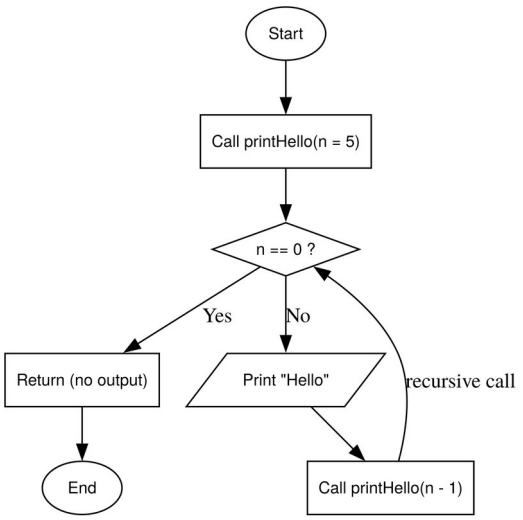
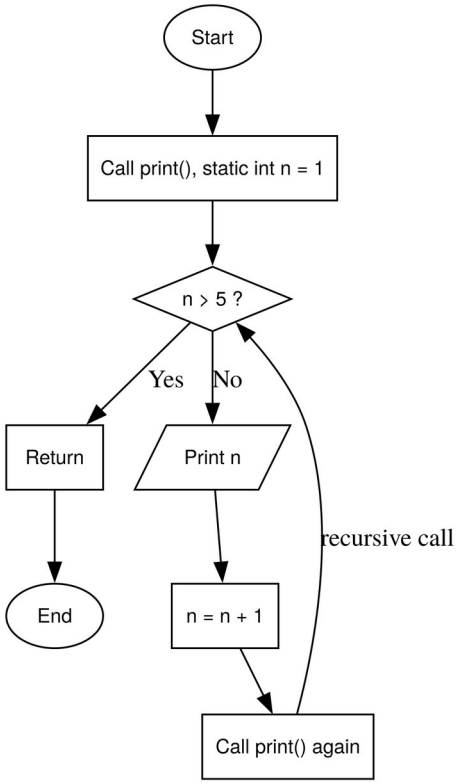
Phase 1 – Recursion Mindset Reset

Question 1



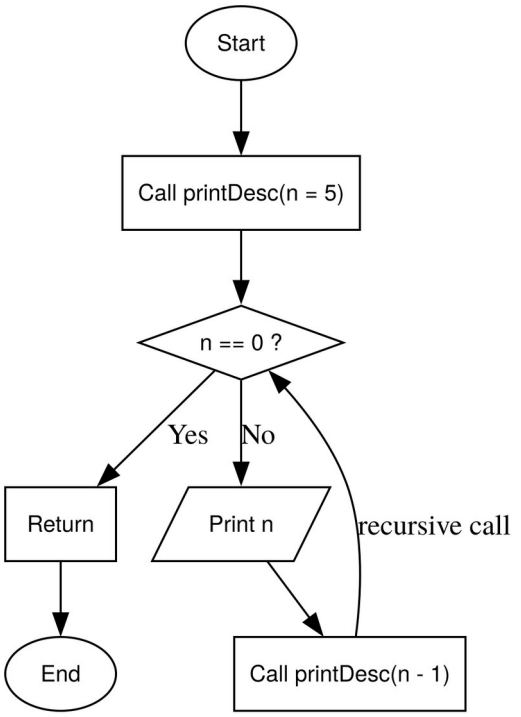

```
void print_hello(int n) {
    // recur pritns hello
    if (n == 0) {
        return ;
    }
    cout << "Hello World!" << endl;
    print_hello(n-1);
}
```

Question 2



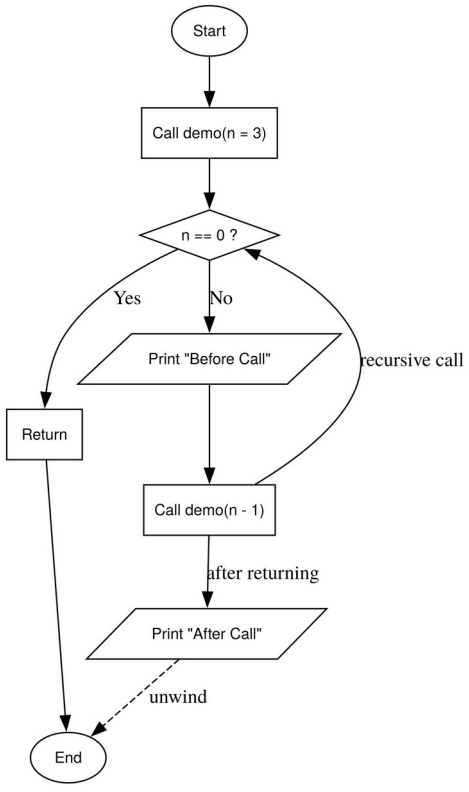

```
void print_1_to_5() {
    static int c = 1;
    if (c > 5) {
        return;
    }
    cout << c << endl;
    c++;
    print_1_to_5();
}
```

Question 3



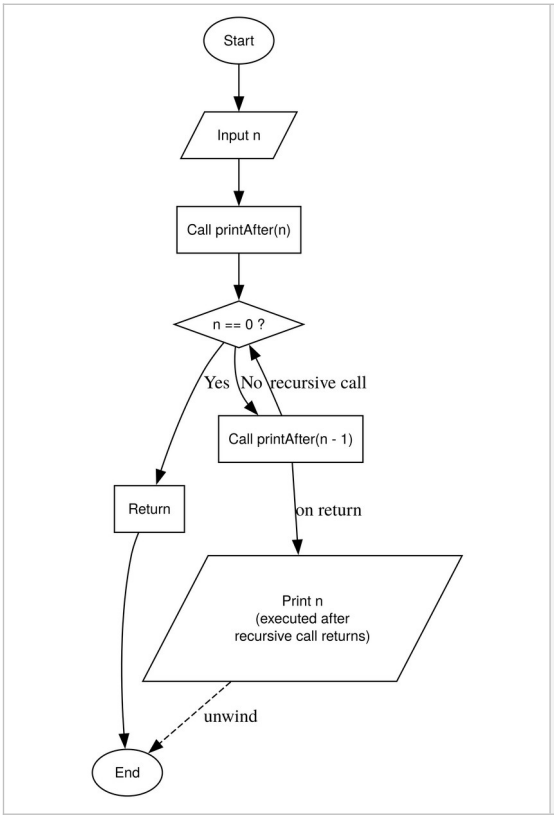

```
void print_5_to_1(int n) {
    if (n < 1) {
        return;
    }
    cout << n << endl;
    print_5_to_1(n - 1);
}
```

Question 4




```
void obeserving_order(int n) {
    if (n < 1) {
        return ;
    }
    cout << "Before call: " << n << endl;
    obeserving_order(n-1);
    cout << "After call: " << n << endl;
}
```

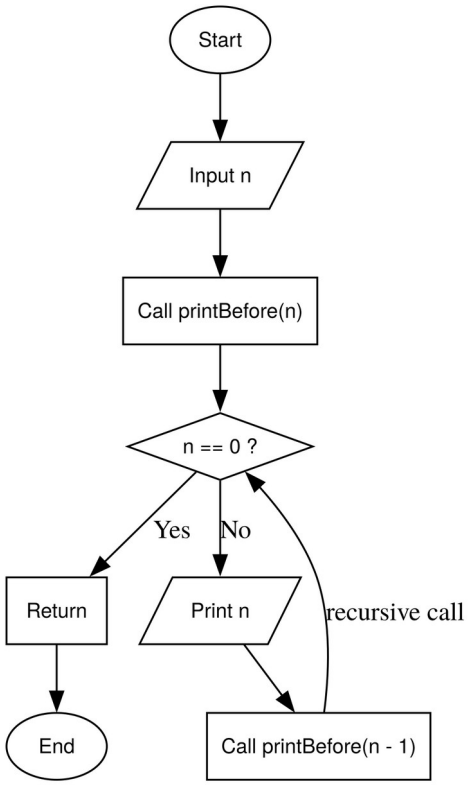
Question 5




```
void print_1_to_n_after(int n) {
    if (n < 1) return ;
    print_1_to_n_after(n-1);
    cout << n << endl;

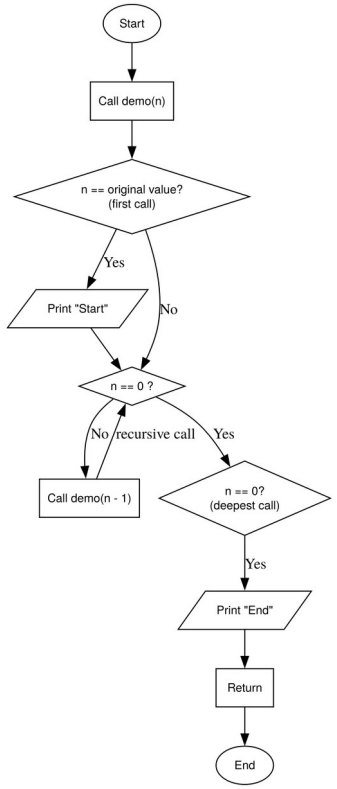
}
```

Question 6



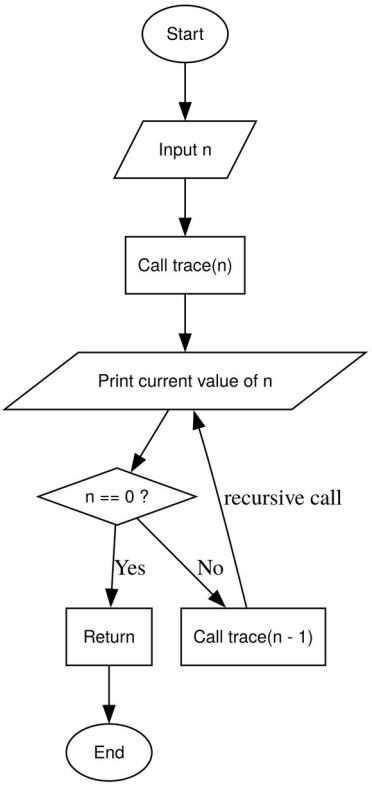

```
void print_1_to_n_before(int n, int curr) {
    if (curr > n) return ;
    cout << curr << endl;
    print_1_to_n_before(n,  curr+1);
}
```

Question 7



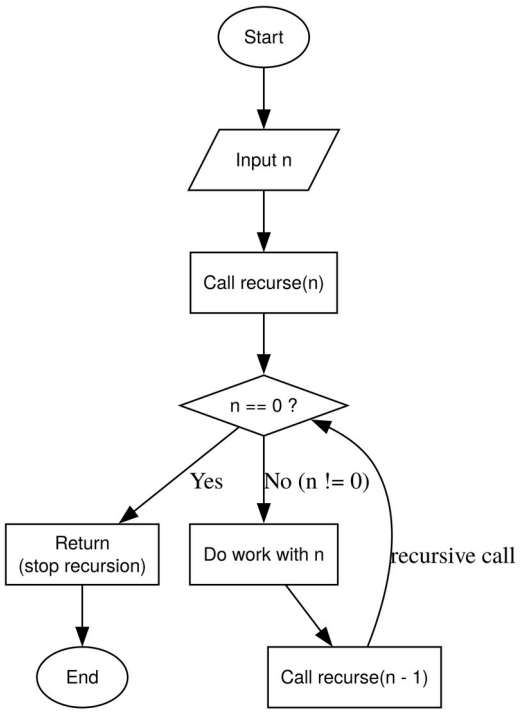

```
void print_start_2_end(int n) {
    static bool is_first = true;
    if (is_first) {
        cout << "Start" << endl;
        is_first = false;
    }
    if (n == 0) {
        cout << "End" << endl;
        return;
    }
    print_start_2_end(n-1);
}
```

Question 8



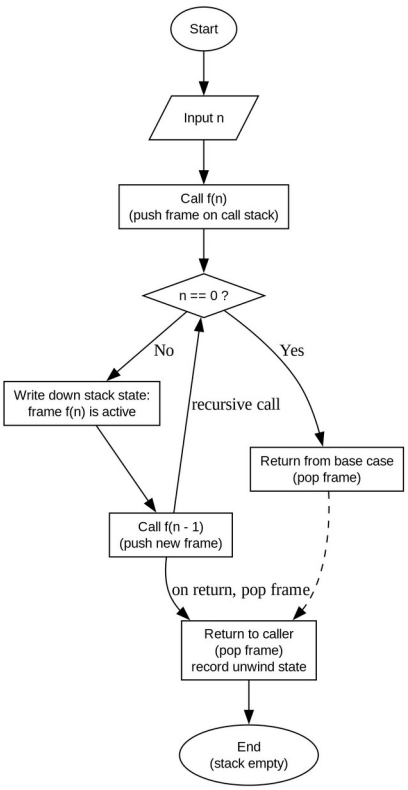

```
void print_current_n(int n) {
    if(n==0) return ;
    cout << "Current n: " << n << endl;
    print_current_n(n-1);
}
```

Question 9




```
void stop_at_zero(int n) {
    if (n == 0) return ;
    stop_at_zero(n-1);
}
```

Question 10



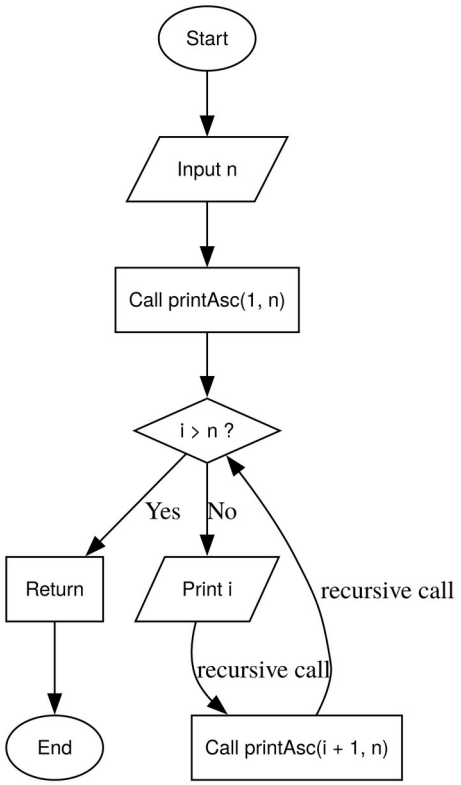
Dry-run for print_5_to_1(5):
Call Stack Pushing:
print_5_to_1(5): Prints 5 -> Calls print_5_to_1(4)
print_5_to_1(4): Prints 4 -> Calls print_5_to_1(3)
print_5_to_1(3): Prints 3 -> Calls print_5_to_1(2)
print_5_to_1(2): Prints 2 -> Calls print_5_to_1(1)
print_5_to_1(1): Prints 1 -> Calls print_5_to_1(0)
print_5_to_1(0): Base case reached (0 < 1) -> Returns

Call Stack Popping:
print_5_to_1(0) popped off stack
print_5_to_1(1) popped off stack
print_5_to_1(2) popped off stack
print_5_to_1(3) popped off stack
print_5_to_1(4) popped off stack
print_5_to_1(5) popped off stack -> Execution finishes

Output:
5
4
3
2
1

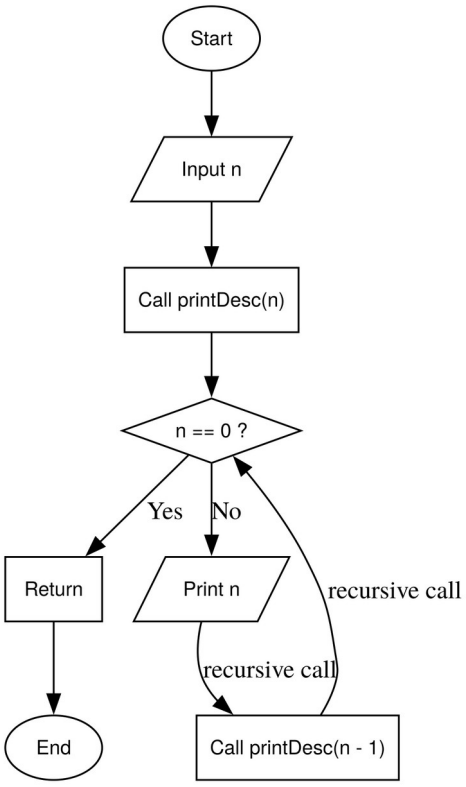
Phase 2 – Linear Number Recursion

Question 1



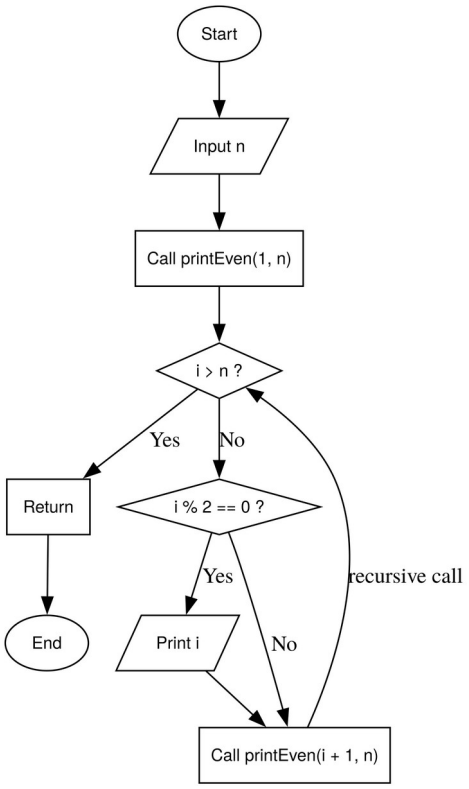

```
void print_1_to_n_recur(int c, int n) {
    if (c> n) return;
    cout << c << endl;
    print_1_to_n_recur(c+1, n);
}
```

Question 2



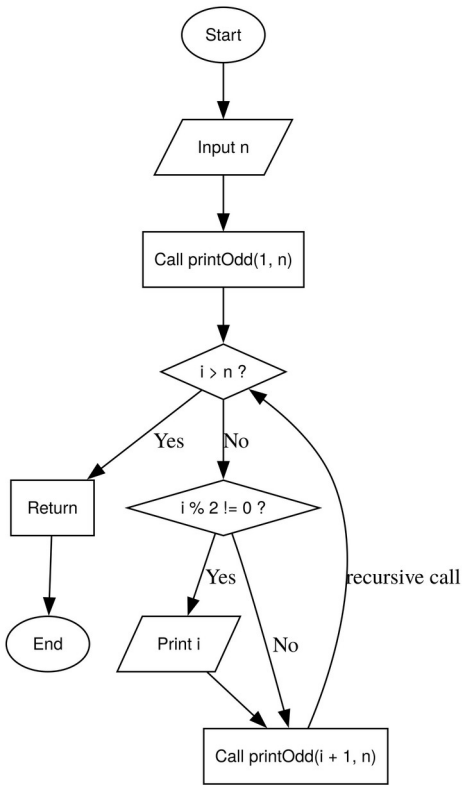

```
void print_n_to_1(int n) {
    if (n == 0) return ;
    cout << n << endl;
    print_n_to_1(n-1);
}
```

Question 3



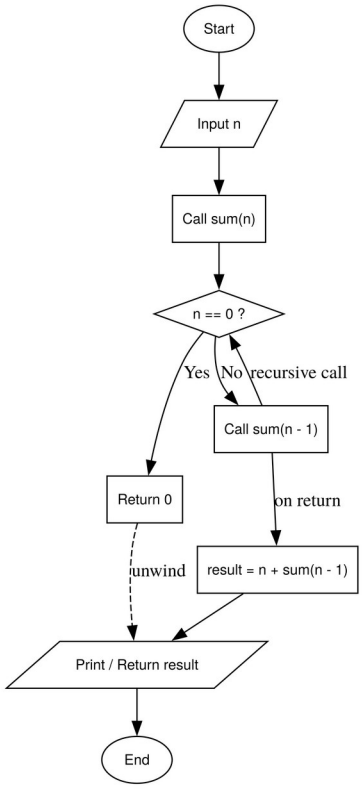

```
void print_1_to_n_recur_even(int c, int n) {
    if (c > n) return;
    if (c % 2 == 0) cout << c << endl;
    print_1_to_n_recur_even(c+1, n);
}
```

Question 4



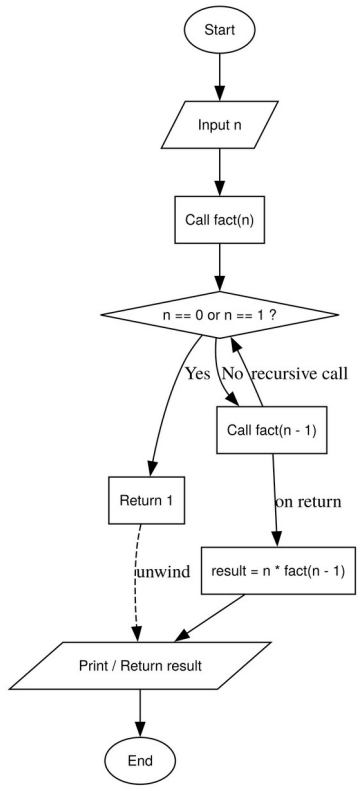

```
void print_1_to_n_recur_odd(int c, int n) {
    if (c > n) return;
    if (c % 2 == 1) cout << c << endl;
    print_1_to_n_recur_odd(c+1, n);
}
```

Question 5



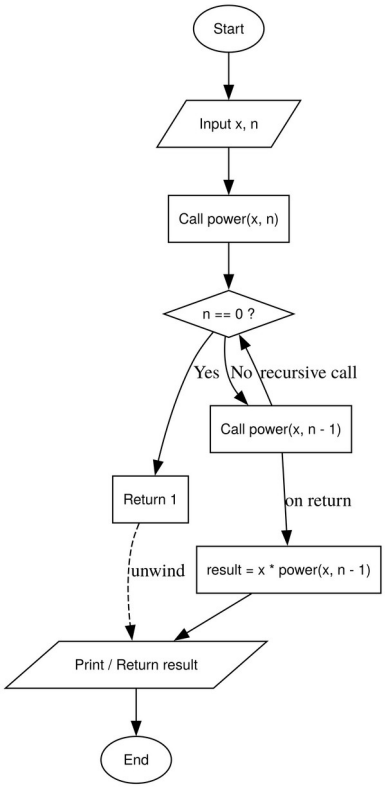

```
ll recur_sum(ll n) {
    if (n <= 0 ) {
        return 0;
    } else {
        return n + recur_sum(n-1);
    }
}
```

Question 6



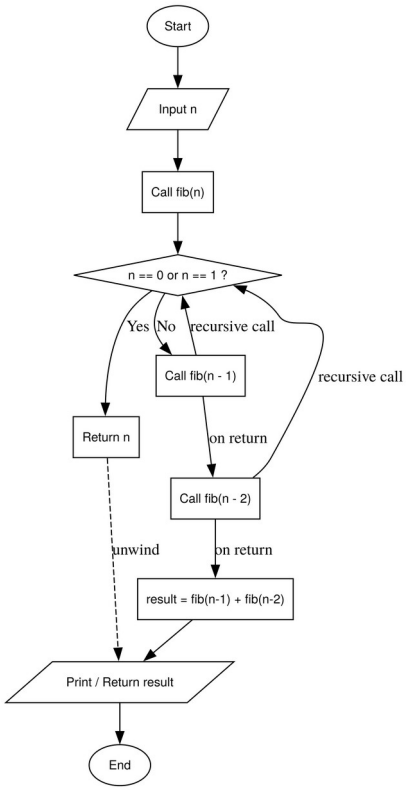

```
ll fact_recur(int n) {
    if (n ==0 || n == 1) {
        return 1;
    } else {
        return n * fact_recur(n-1);
    }
}
```

Question 7



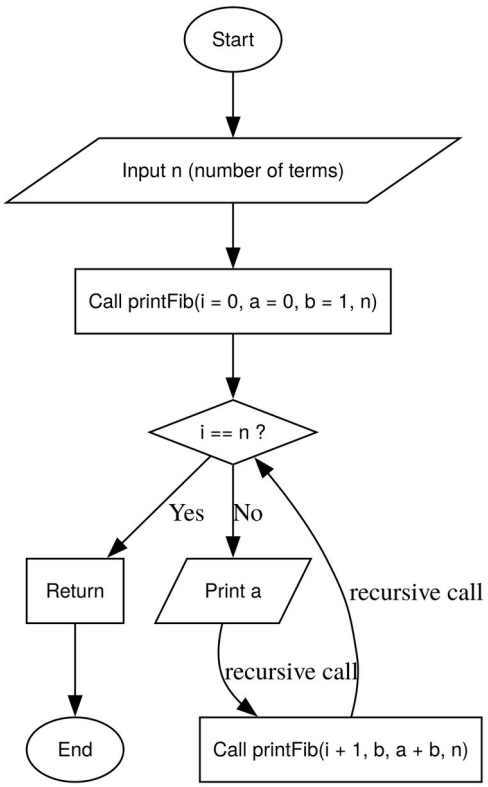

```
ll calc_power(int x, int n) {
    if (n ==0) return 1;
    return x * calc_power(x, n-1);
}
```

Question 8




```
ll fibonacci(int n) {
    if (n  == 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    }
    else {
        return  fibonacci(n-1) + fibonacci(n-2);
    }
}
```

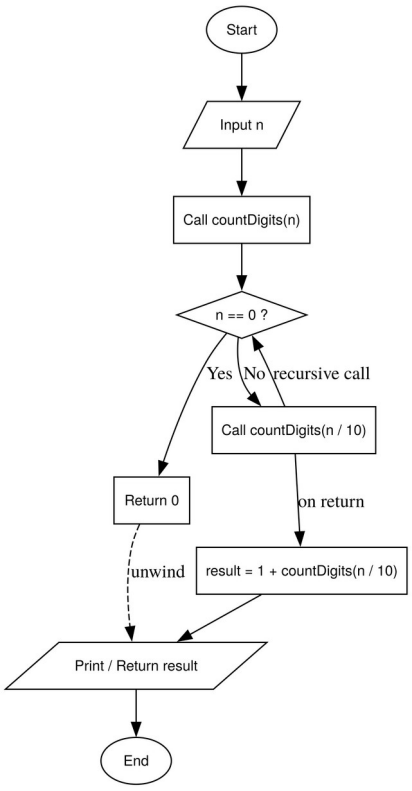
Question 9




```
ll fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}

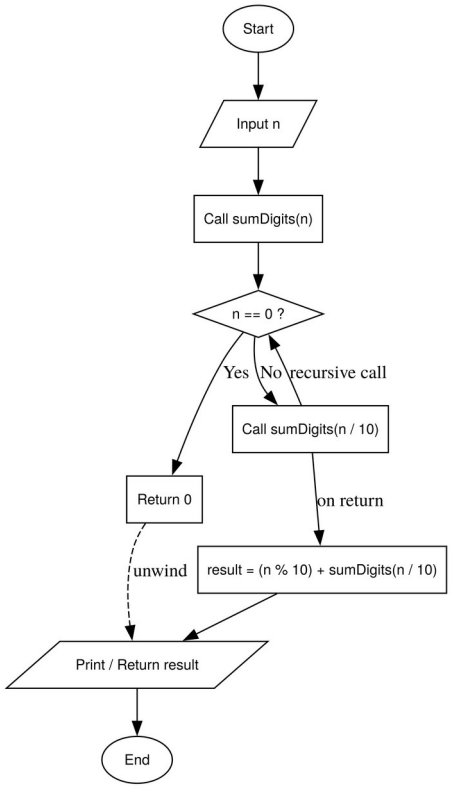
void print_fibbonaci(int n, int i = 0) {
    if (i >= n) {
        cout << endl;
        return;
    }
    cout << fibonacci(i) << " ";
    print_fibbonaci(n, i+1);
}
```

Question 10



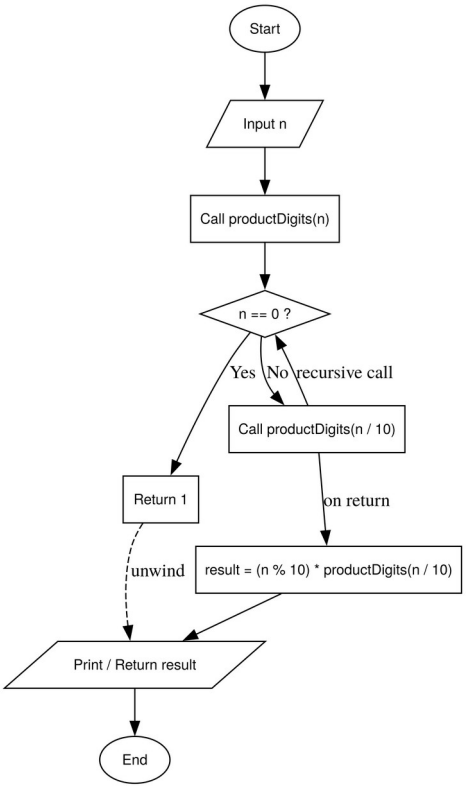

```
int count_digits(ll n) {
    if (n < 10) return 1;
    return 1 + count_digits(n/10);
}
```

Question 11




```
int sum_digits(int n) {
    if (n == 0) return 0;
    return (n % 10) + sum_digits(n/10);
}
```

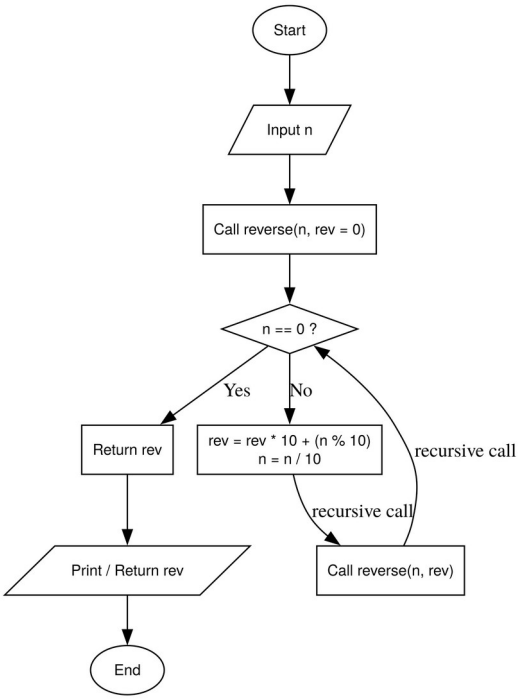
Question 12




```
int prod_digits(int n) {

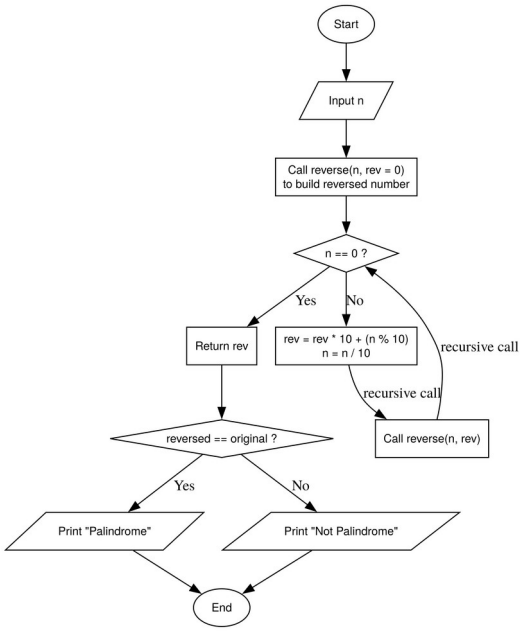
    if (n == 0) return 1;
    return (n % 10) * prod_digits(n/10);
}
```

Question 13




```
ll rev_num(ll n, ll rev = 0) {
    if (n == 0) return rev;
    return rev_num(n / 10, (rev * 10) + (n % 10));
}
```

Question 14




```
bool palindrome(ll num, int left, int right=-1) {
    // if num is neg, return false
    if (num < 0) {
        return false;
    }
    if ((num <= 9 && num >= 0)) {
        return true;
    }

    string s = to_string(num);

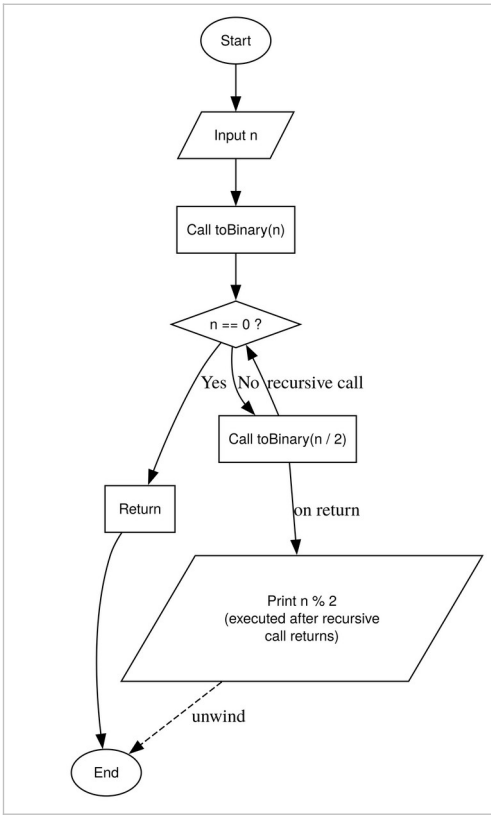
    if (right == -1) {
        right = s.length() -1;
    }

    // base case if the pointers cross or meet, every digit is
matched then
    if (left >= right) return true;

    // if digits at current pointers dont match., then not a
match
    if (s[left] != s[right]) {
        return false;
    }

    // pointers returned ,but number retained
   return palindrome(num, left+1, right-1);
}
```

Question 15

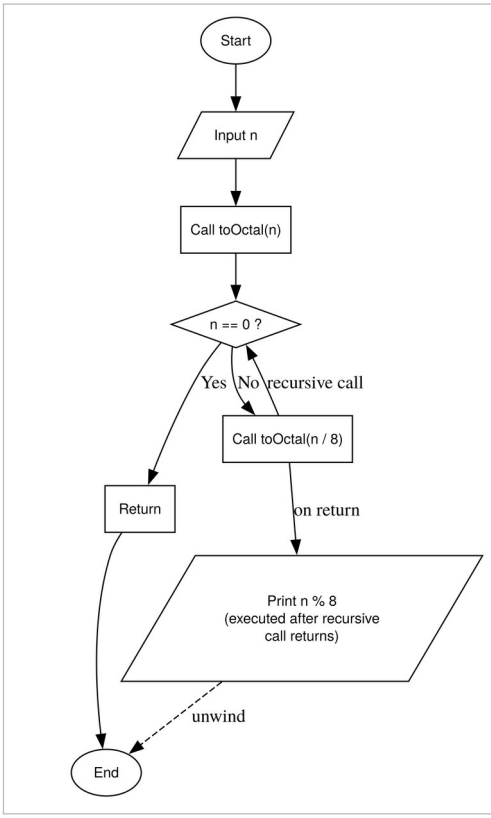



```
string decimal_bin(ll n){

    // base case when n == 0;
    if (n==0) return  "0";
    if (n==1) return "1";

    return decimal_bin(n/2) + (char)((n%2) + '0');
}
```

Question 16

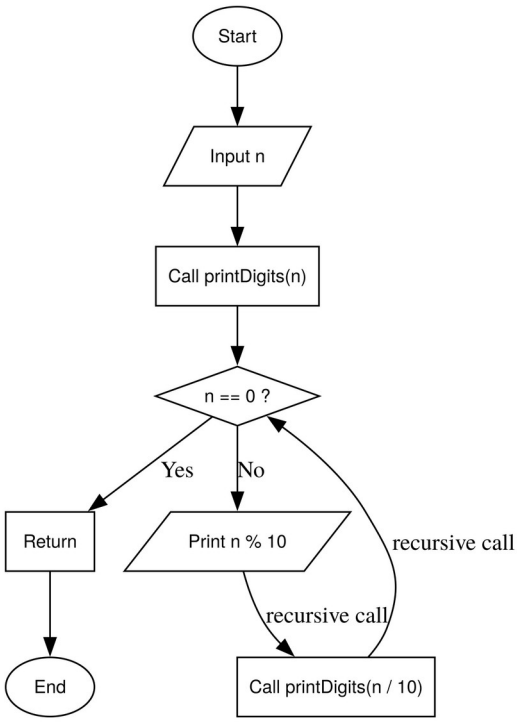



```
string oct_deci(ll n) {
    // base case when == 0
    if (n==0) return "0";

    // return remainder if n < 8
    if (n < 8) return to_string(n);

    return oct_deci(n / 8) + (char)((n % 8) + '0');
}
```

Question 17

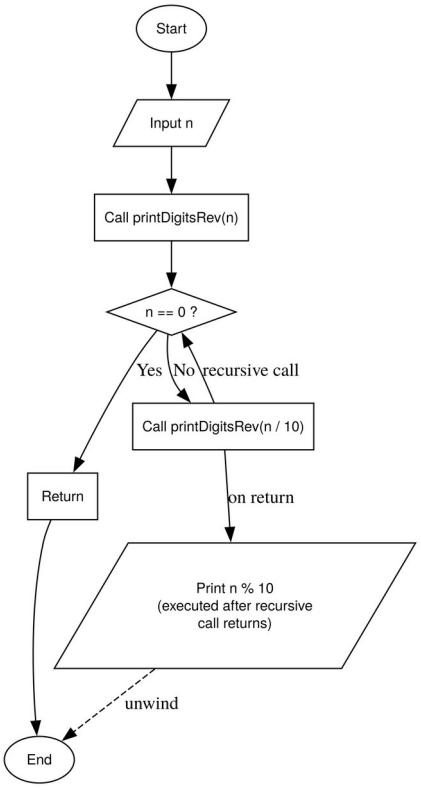



```
void print_digit_normal(ll n) {
    if (n < 0)  n = -n;
    if (n==0) return;

    // gcc is forced to recurisvely call this formula till base
case. and then pritns num
    print_digit_normal(n / 10);
    cout << (n%10) << endl;

}
```

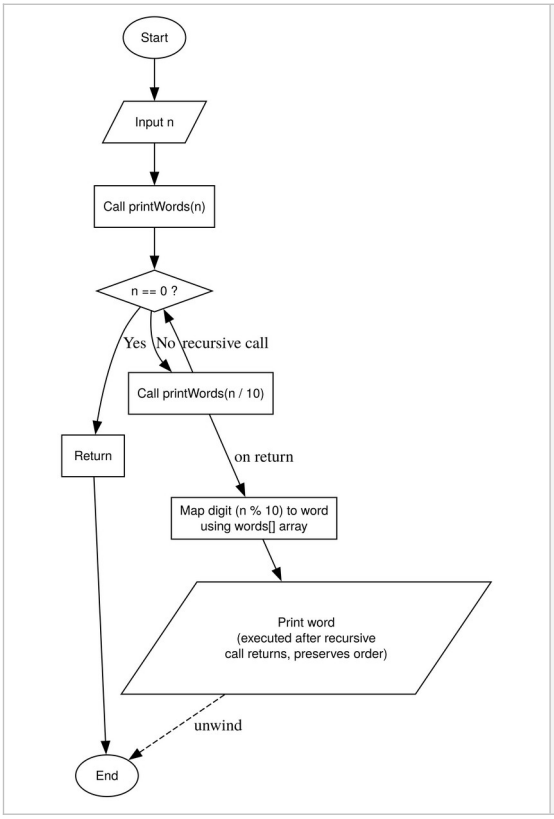
Question 18




```
void print_digit_rev(ll n) {
    if (n < 0) n = -n;
    // base case
    if (n == 0) return;

    cout << (n%10) << endl;
    print_digit_rev(n/10);
}
```

Question 19




```
void print_word(int n) {
switch (n) {
    case 0: cout << "Zero "; break;
    case 1: cout << "One "; break;
    case 2: cout << "Two "; break;
    case 3: cout << "Three "; break;
    case 4: cout << "Four "; break;
    case 5: cout << "Five "; break;
    case 6: cout << "Six "; break;
    case 7: cout << "Seven "; break;
    case 8: cout << "Eight "; break;
    case 9: cout << "Nine "; break;
} }

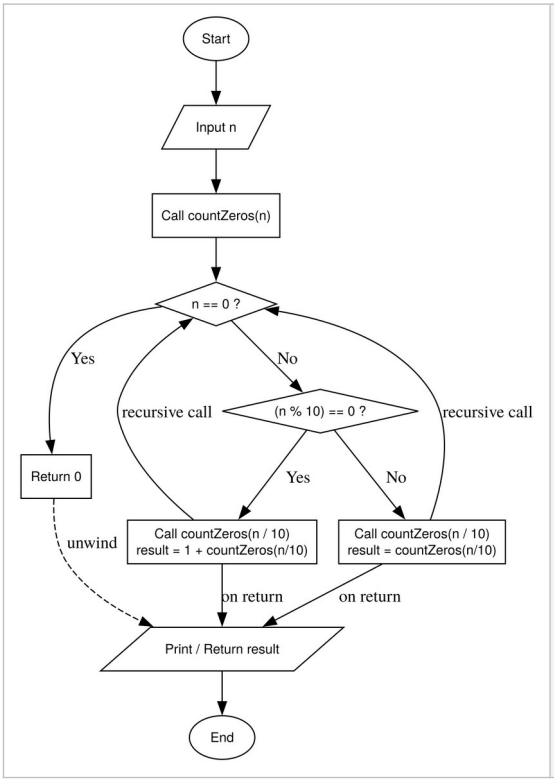
void print_digits_words(ll n) {
    if (n==0) return;

    // recurive call
    print_digits_words(n/10);

    // prints word up  the stack
    print_word(n % 10);

}
```

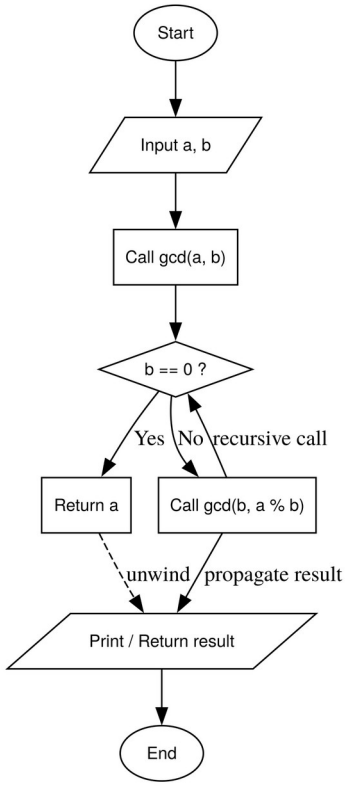
Question 20




```
int count_zero_recur(ll n) {
    if (n < 0) n = -n;
    if (n < 10) return (n == 0) ? 1 : 0;
    return ((n % 10 == 0) ? 1 : 0) + count_zero_recur(n/10);
}
```

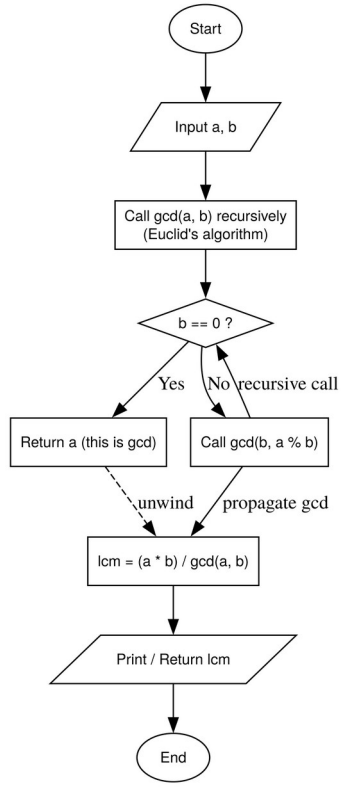
Phase 3 – Mathematical Thinking With Recursion

Question 1




```
ll gcd_euclid(ll a, ll b) {
    // base case
    if (b == 0)  return a;
    return gcd_euclid(b, (a % b));
}
```

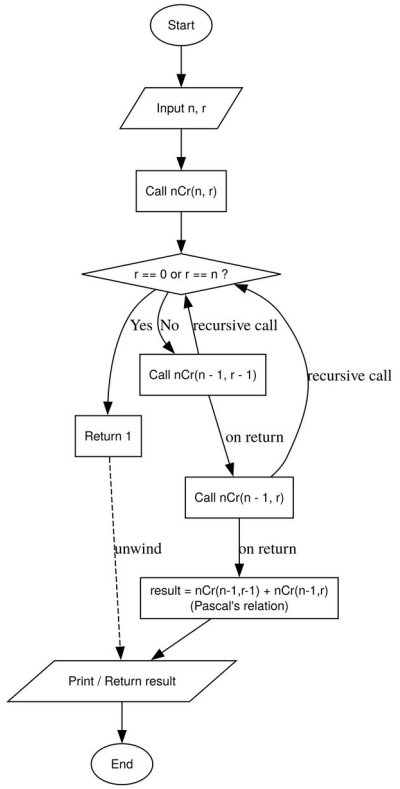
Question 2




```
ll gcd_euclid(ll a, ll b) {
    if (b == 0) return a;
    return gcd_euclid(b, a % b);
}

ll lcm(ll a, ll b) {
    a = abs(a);
    b = abs(b);
    if (a == 0 || b == 0) return 0;
    return (a / gcd_euclid(a, b)) * b;
}
```

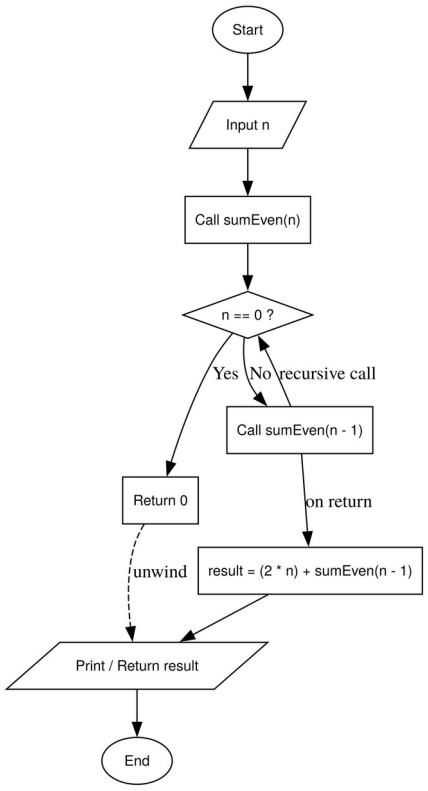
Question 3




```
ll ncr_recursive(ll n, ll r) {
    // base cases
    if (r > n) return 0;
    if (r== 0 || r == n) return 1;

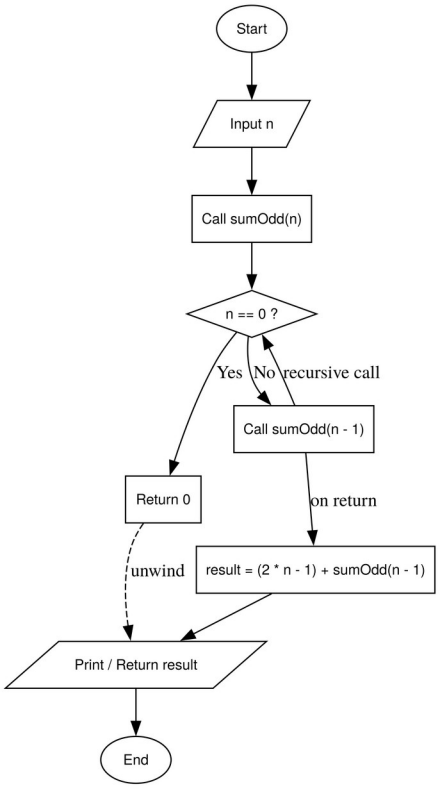
    return ncr_recursive(n-1, r - 1) + ncr_recursive(n-1,  r);
}
```

Question 4



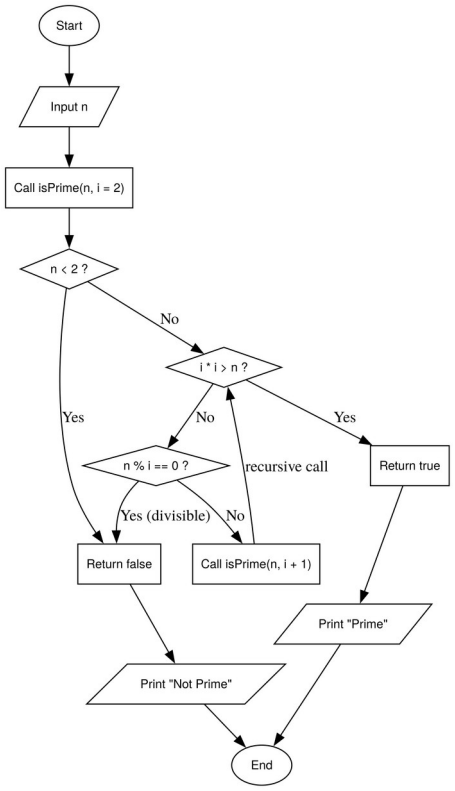

```
ll sum_first_n_even(ll n) {
    if (n==0) return 0;
    return (2*n) + sum_first_n_even(n-1);
}
```

Question 5




```
ll sum_first_n_odd(ll n) {
    if (n==0) return 0;
    return (2 * n  - 1) + sum_first_n_odd(n-1);
}
```

Question 6



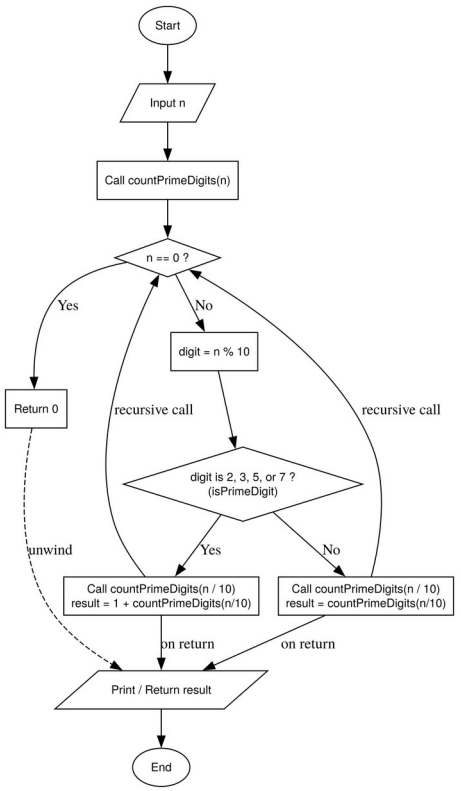

```
bool is_prime_recursive(ll n, ll divisor = 2) {
    // first base case
    if (n <= 1) return false;

    // second base case
    if (divisor * divisor > n) return true;

    //3rd base case when a num is a divisor
    if (n % divisor == 0) return false;

    // Recursive Step: Increment divisor and check again
    return is_prime_recursive(n, divisor + 1);
}
```

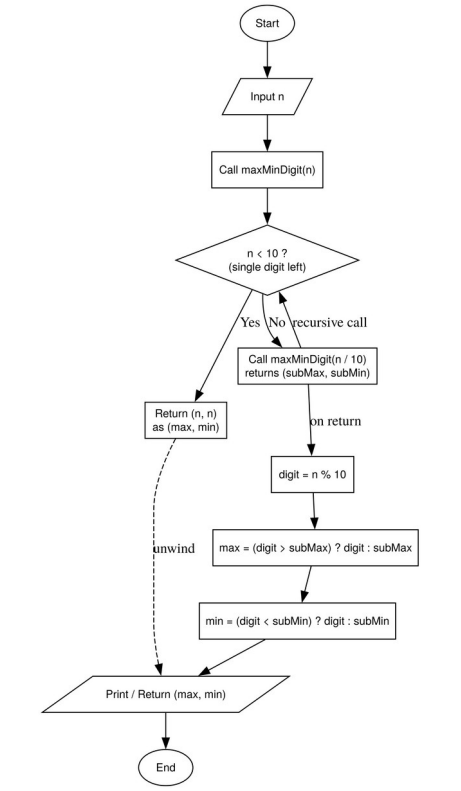
Question 7




```
int count_prime(ll n) {
    if (n == 0) return 0;
    int dig = n % 10;

    if (is_prime_recursive(dig)) return 1 + count_prime(n / 10);
    return 0 + count_prime(n / 10);
}
```

Question 8




```
void find_min_max(int n, int &min, int &max){
    static int min_digit = 9;
    static int max_digit = 0;

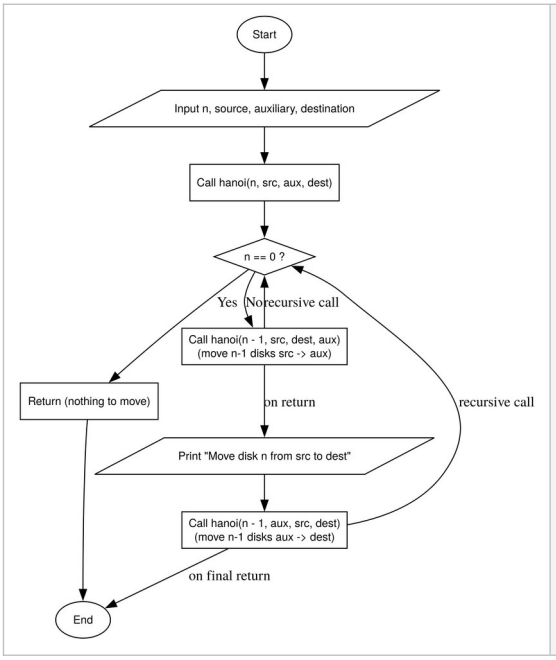
    // base case
    if (n == 0){
        // copy static results to the reference varialbes
        min = min_digit;
        max = max_digit;

        // resets static variables
        min_digit = 9;
        max_digit = 0;
        return;
    }

    int current = n % 10;

    if (current < min_digit) min_digit =current;
    if (current > max_digit) max_digit = current;
    find_min_max(n/10, min, max);
}
```

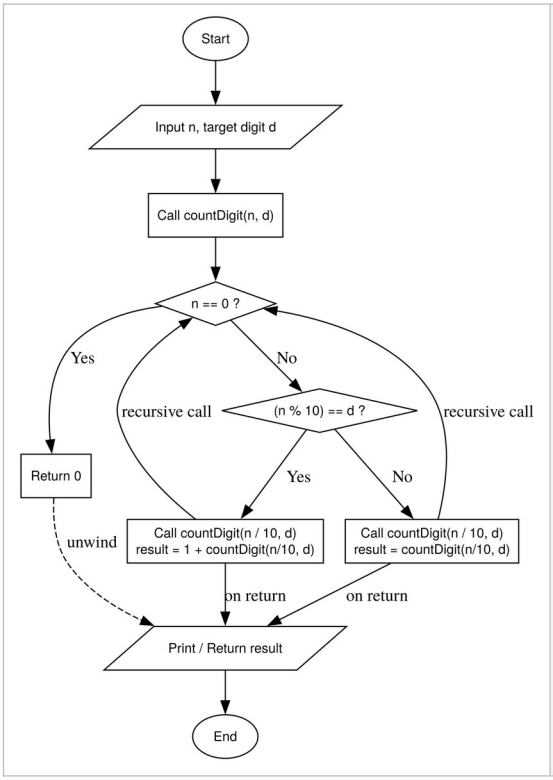
Question 9




```
void tower_of_hanoi_problem(int n, char from_rod, char to_rod,
char aux_rod) {
    if (n == 0) return;

    tower_of_hanoi_problem(n-1, from_rod, aux_rod, to_rod);
    cout << "moving disk " << n << " from rod " << from_rod << "
to rod " << to_rod << endl;
    tower_of_hanoi_problem(n-1, aux_rod, to_rod, from_rod);
}
```

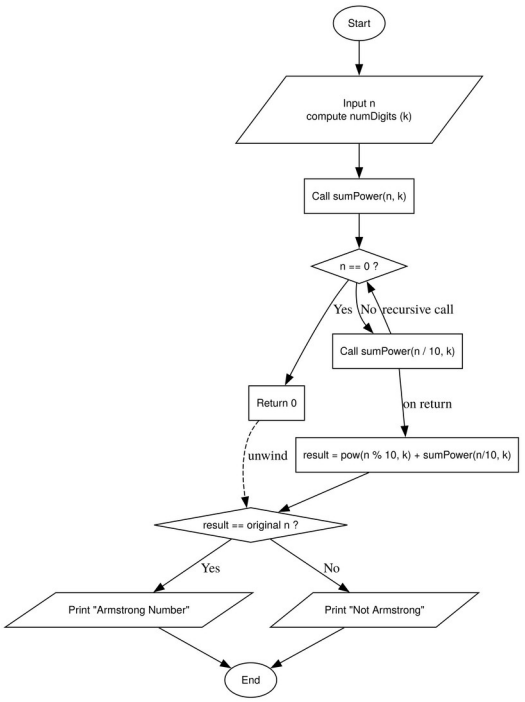
Question 10




```
int count_occurances(ll n, int digit) {
    if (n==0) return 0;
    int dig = n % 10; //gets the last digit

    if (dig == digit) return 1 + count_occurances(n / 10,
digit);
    else return 0+ count_occurances(n/ 10, digit);
}
```

Question 11




```
bool check_armstrong(int n, int orig_num, int k, int sum) {

    // base case
    if (n == 0) return (orig_num == sum);

    int digit = n % 10;
    sum += pow(digit, k);

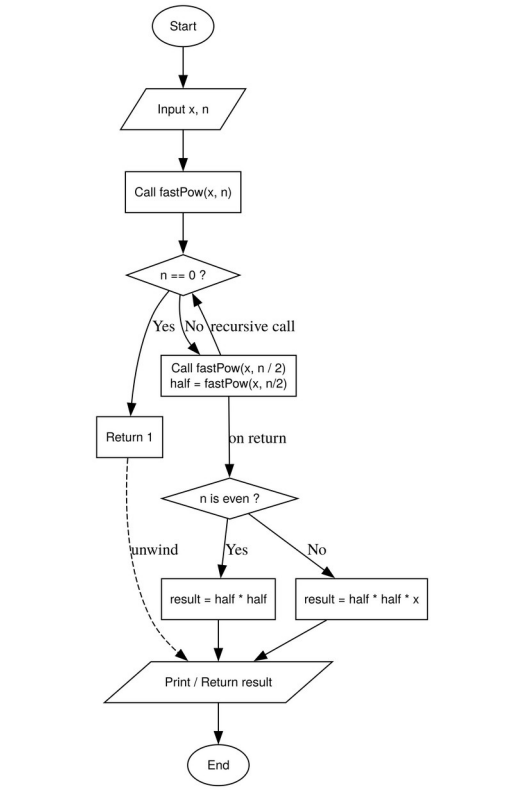
    return check_armstrong(n/10,  orig_num,  k,  sum);
}

int count_digit(int n) {
    // this to find k
    if (n==0) return 0;
    return 1+count_digit(n/10);
}

bool is_armstrong(ll n) {
    if (n < 0) return false;
    if (n == 0) return true;

    return check_armstrong(n, n, count_digit( n), 0);
}
```

Question 12

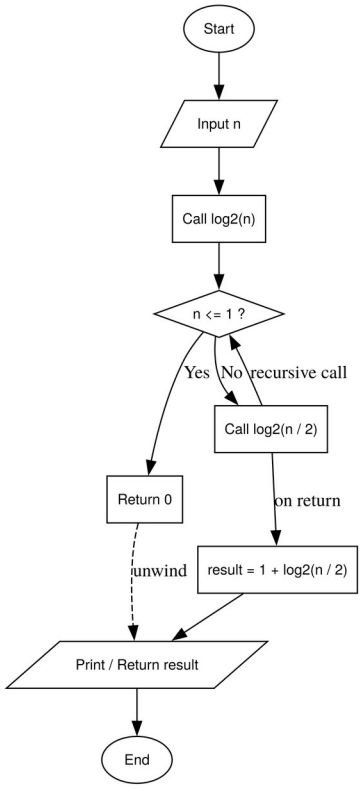



```
ll exponentiation_by_sq(ll n, int k) {
    if (k == 0) return 1;

    ll half_power = exponentiation_by_sq(n, k /2);

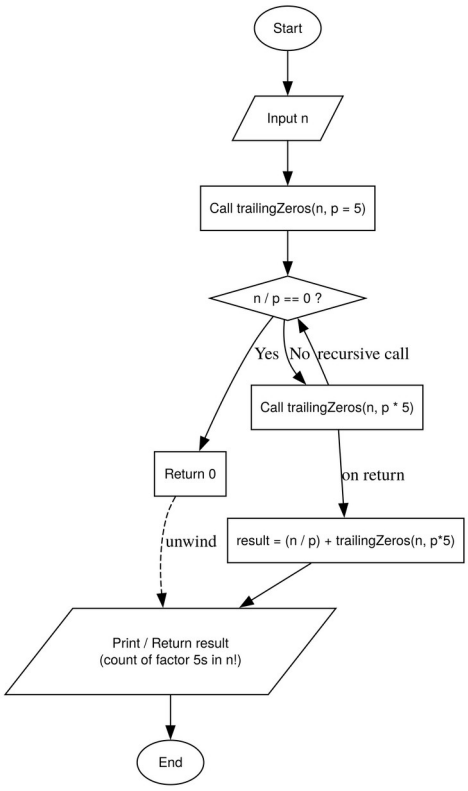
    if(k % 2 == 0) {
        return half_power * half_power;
    }
    else {
        return n * half_power * half_power;
    }
}
```

Question 13




```
int log2_recursive(int n) {
    if (n <= 1) return 0;
    return 1 + log2_recursive(n/2);
}
```

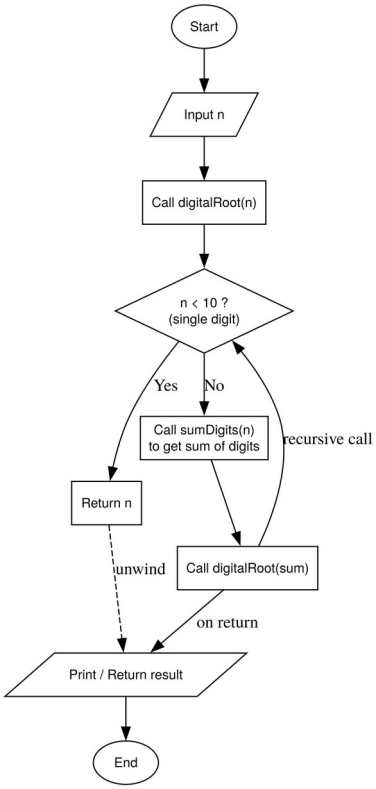
Question 14




```
ll count_trailing_zeros(ll n) {
    if (n < 5) return 0;

    return (n / 5) + count_trailing_zeros(n/5);
}
```

Question 15

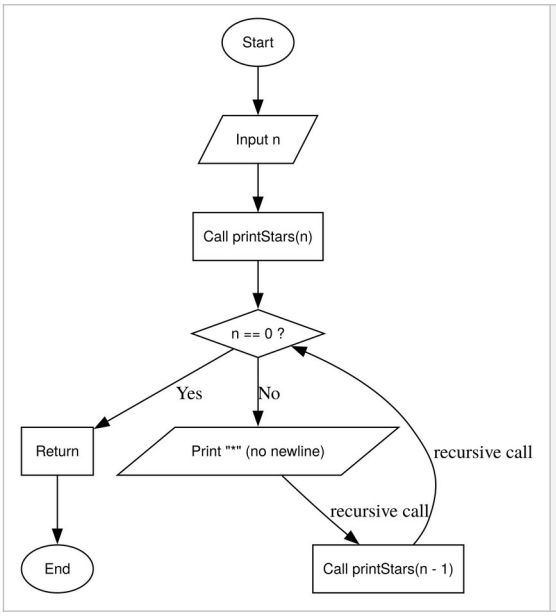



```
int sum_digits(int n) {
    if (n==0) return 0;
    return (n % 10) + sum_digits(n/10);
}

int digital_root(int n) {
    if (n < 10) return n;
    int sum = sum_digits(n);
    return digital_root(sum);
}
```

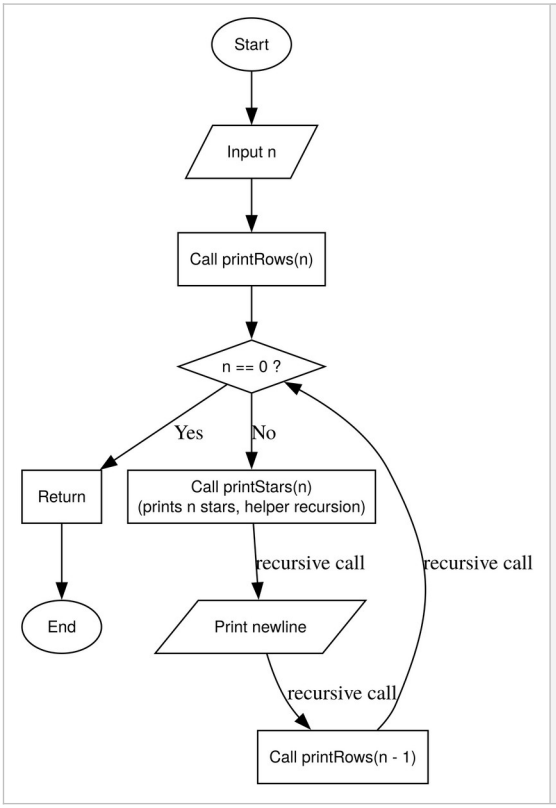
Phase 4 – Controlled Printing & Patterns

Question 1




```
void print_line(ll n) {
    if (n == 0)  {cout << endl; return ; }
    cout << "*";
    print_line(n-1);
}
```

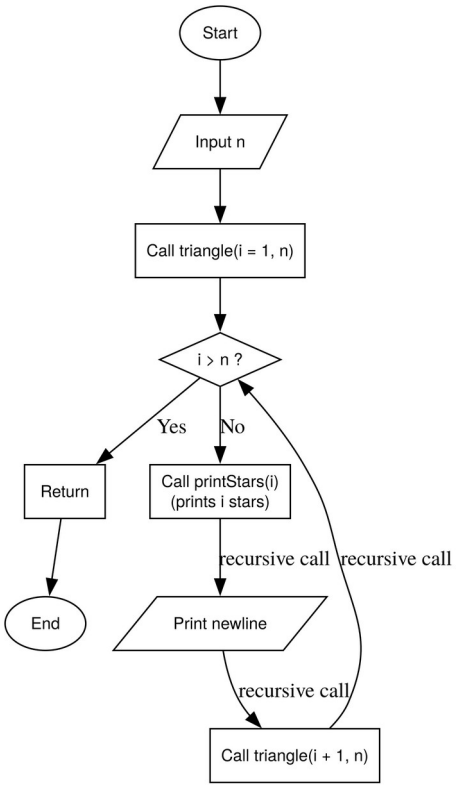
Question 2




```
void print_line(ll n) {
    if (n == 0) {
        cout << endl;
        return;
    }
    cout << "*";
    print_line(n-1);
}

void print_sq(int n, int current_row = 0) {
    if (current_row >= n) return;
    print_line(n);
    print_sq(n, current_row + 1);
}
```

Question 3



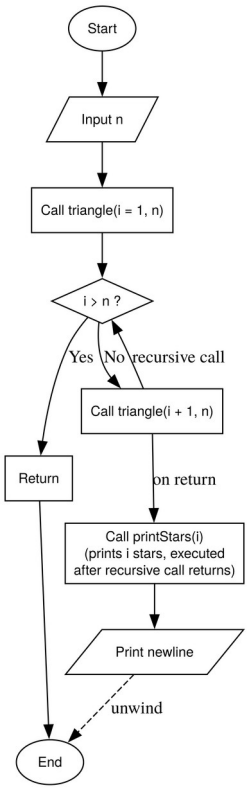

```
void print_row(int n) {
    if (n == 0) return;
    cout << "*";
    print_row(n-1);
}

void print_top_down_triangle(int n, int current_row = 1) {
    if (current_row > n) return;

    // printing statrs for current row
    print_row(current_row); cout << endl;

    print_top_down_triangle(n, current_row+1);
}
```

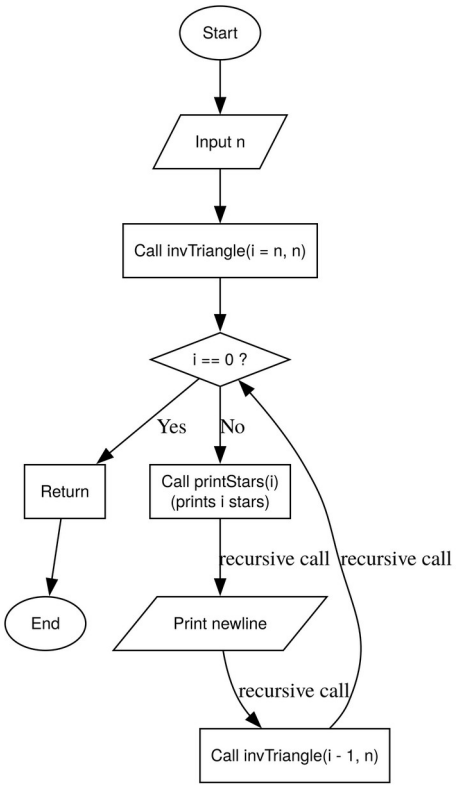
Question 4




```
void print_bottom_up_triangle(int n, int current_row = 1) {
    if (current_row > n) return;

    print_bottom_up_triangle(n, current_row + 1);
    print_row(current_row);
    cout << endl;
}
```

Question 5




```
void print_row(int n) {
    if (n == 0) return;
    cout << "*";
    print_row(n-1);
}

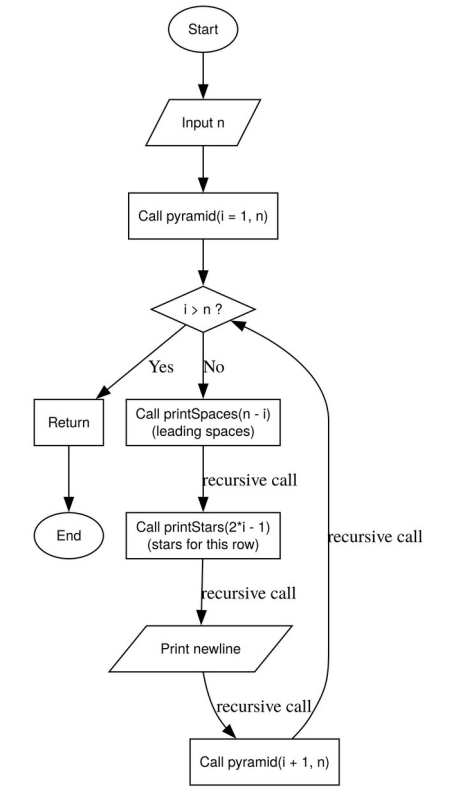
void print_inverted_triangle(int n, int cur) {

    if (cur <= 0) return;


    print_row(cur);
    cout << endl;

    print_inverted_triangle(n, cur - 1);
}
```

Question 6



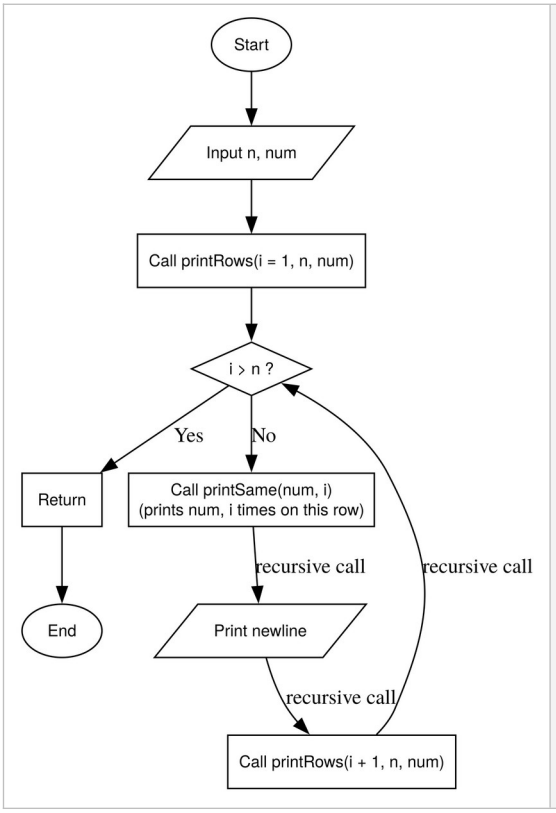

```
void print_space(int n) {
    if (n == 0) return;
    cout << " ";
    print_space(n-1);
}

void print_row(int n) {
    if (n == 0) return;
    cout << "*";
    print_row(n-1);
}

void print_pyramid(int n, int i = 1) {
    if (i > n) return;

    print_space(n - i);
    print_row(2 * i - 1);
    cout << endl;
    print_pyramid(n, i + 1);
}
```

Question 7




```
void print_number(int num, int cols) {
    if (cols <= 0) return;

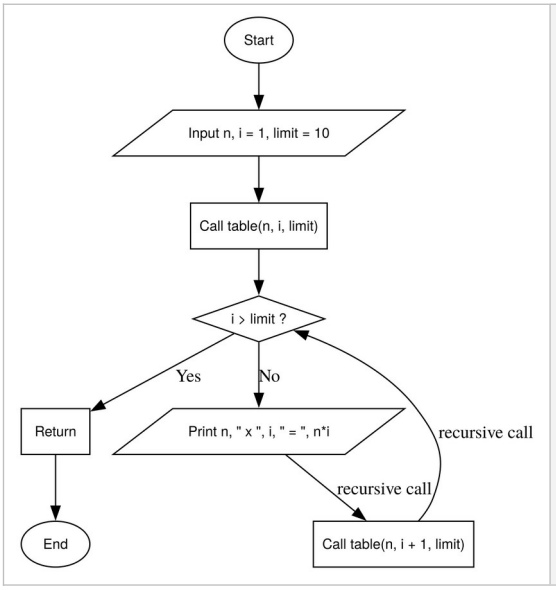
    cout << num << " ";
    print_number(num, cols-1);
}

void print_num_grid(int n, int current_row = 1) {
    if (current_row > n) return;

    print_number(current_row, n);
    cout << endl;

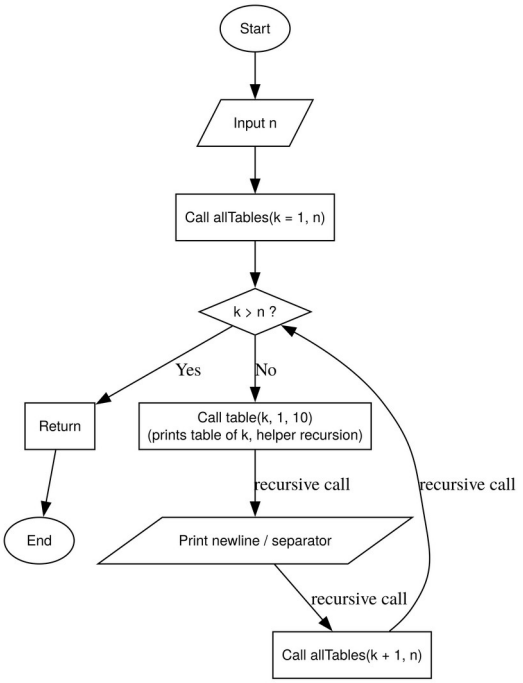
    print_num_grid(n, current_row+1);
}
```

Question 8




```
void print_mult_table(int n, int mut = 12)  {
    if (mut == 0)  return;
    print_mult_table(n, mut - 1);
    cout << n << "* " << mut << "= " << n*mut << endl;
}
```

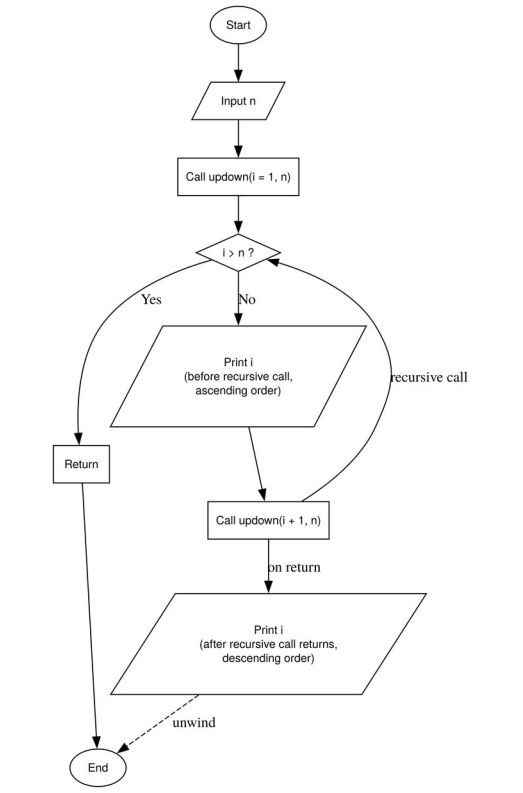
Question 9




```
void print_all_tables(int n, int k = 1) {
    if (k > n) return;

    print_mult_table(k, 10);
    cout << "----------" << endl;
    print_all_tables(n, k + 1);
}
```

Question 10



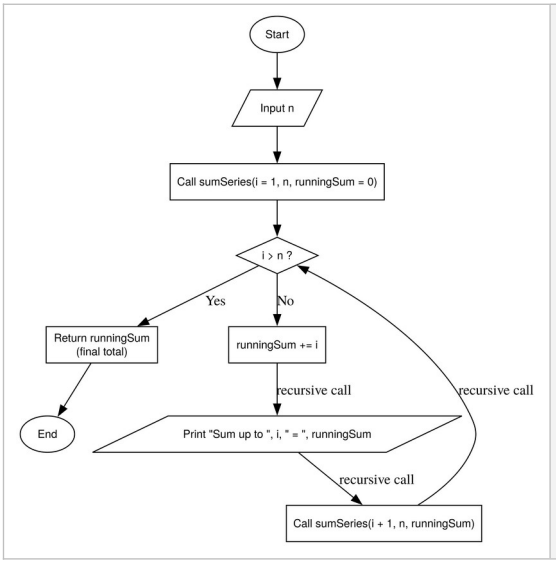

```
void inc_then_dec(int n) {
    if (n <= 0) return;

    cout << n << " ";

    inc_then_dec(n-1);

    cout << n << " ";
}
```

Question 11

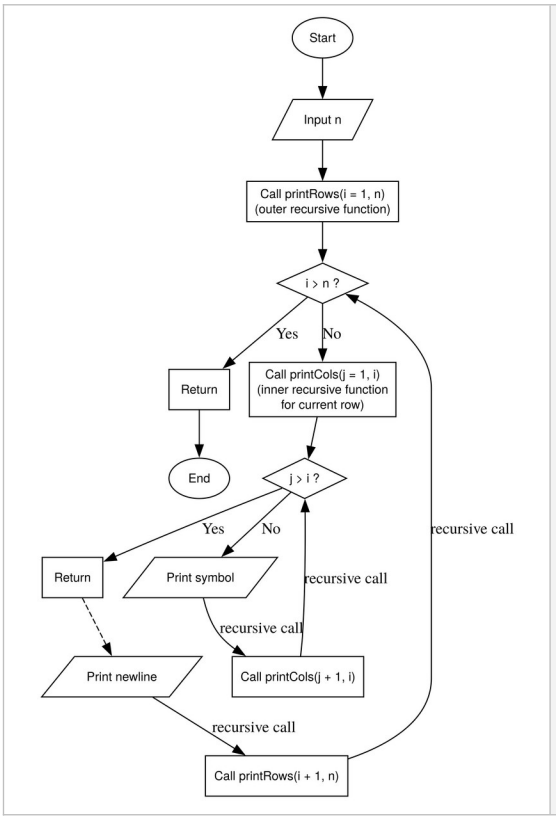



```
ll print_sum_series(ll n) {
    if (n == 0) return 0;

    ll  prev_sum = print_sum_series(n-1);
    ll currnet = prev_sum + n;

    cout << "n: " << n << "-> current sum: " << currnet << endl;
    return currnet;
}
```

Question 12



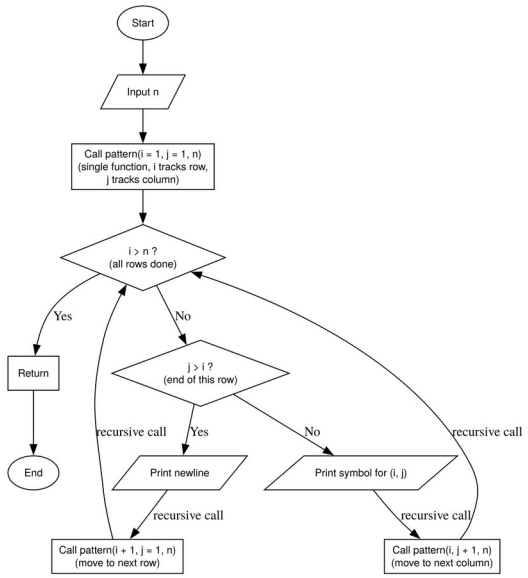

```
void print_cols(int n) {
    if (n <= 0) return;

    cout << "*";
    print_cols(n-1);
}

void print_pattern(int n) {
    // this prints an inverted right angle triangle
    if (n == 0) return;

    print_cols(n); cout << endl;
    print_pattern(n-1);
}
```

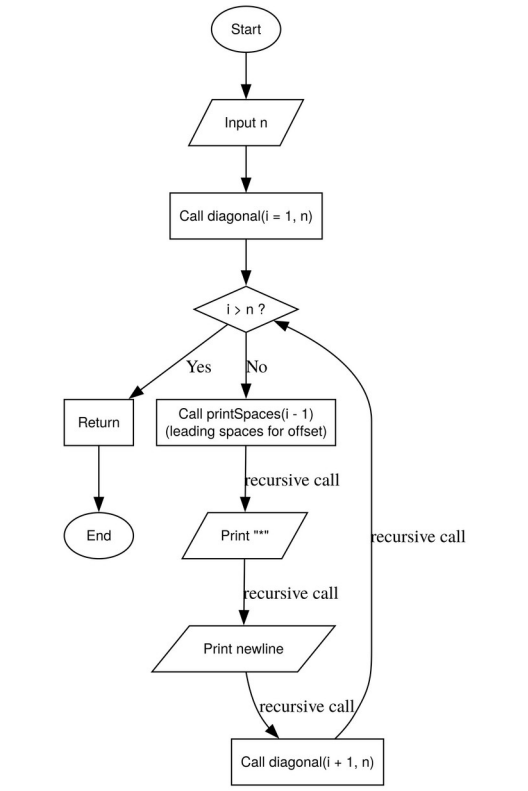
Question 13




```
void print_triangle(int n, int curr_row = 1, int curr_col = 1) {
    if (curr_row > n) return;

    if (curr_col <= curr_row) {
        cout << "*";
        print_triangle(n, curr_row, curr_col + 1);
    }
    else {
        cout << endl;
        print_triangle(n, curr_row + 1, 1);
    }
}
```

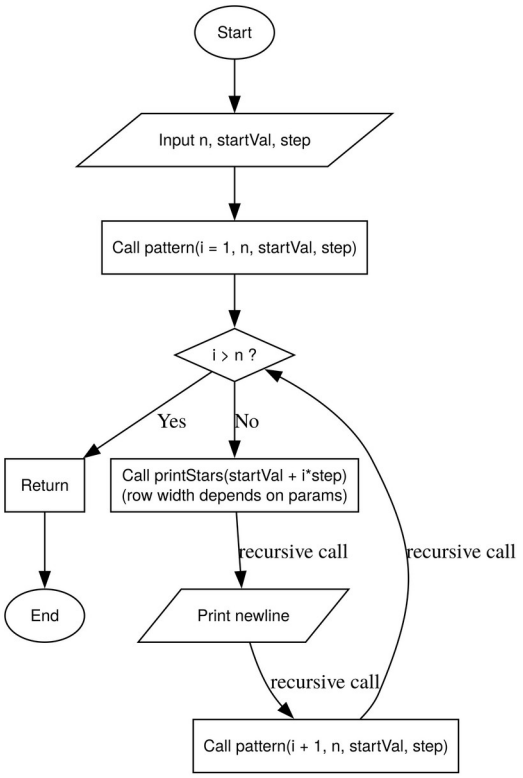
Question 14




```
void print_diagonal(int n, int i = 1) {
    if (i > n) return;

    print_space(i - 1);
    cout << "*" << endl;
    print_diagonal(n, i + 1);
}
```

Question 15

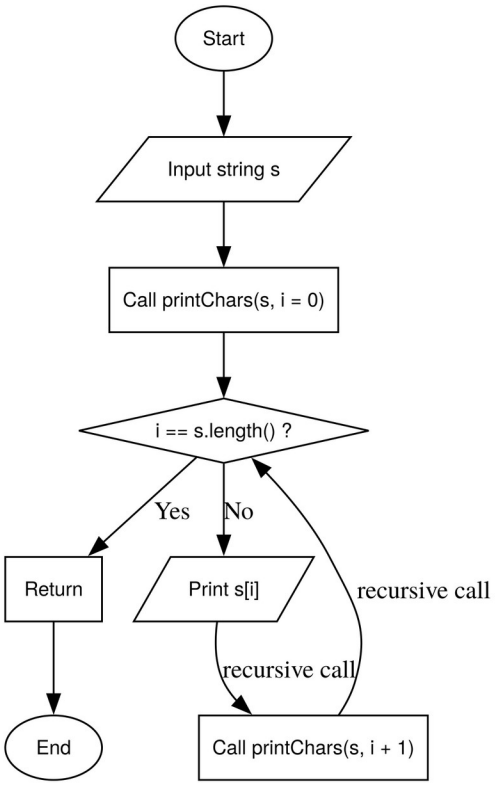



```
void print_param_pattern(int n, int start_val, int step, int i =
1) {
    if (i > n) return;

    print_row(start_val + i * step);
    cout << endl;
    print_param_pattern(n, start_val, step, i + 1);
}
```

Phase 5 – String Recursion (Index Based Thinking)

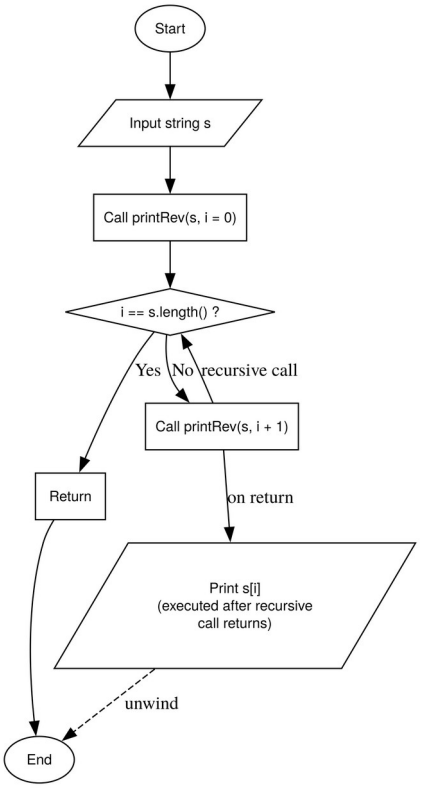
Question 1




```
void print_string(string s, int n) {
    if(n == 0) return ;

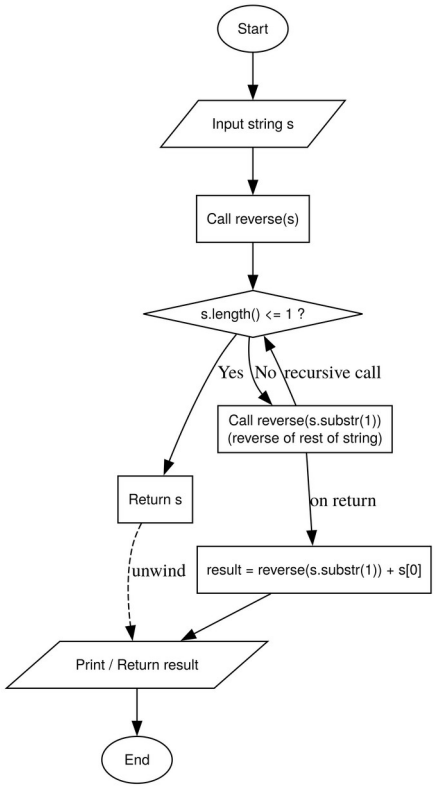
    print_string(s, n-1);
    cout << s[n-1] << endl;
}
```

Question 2



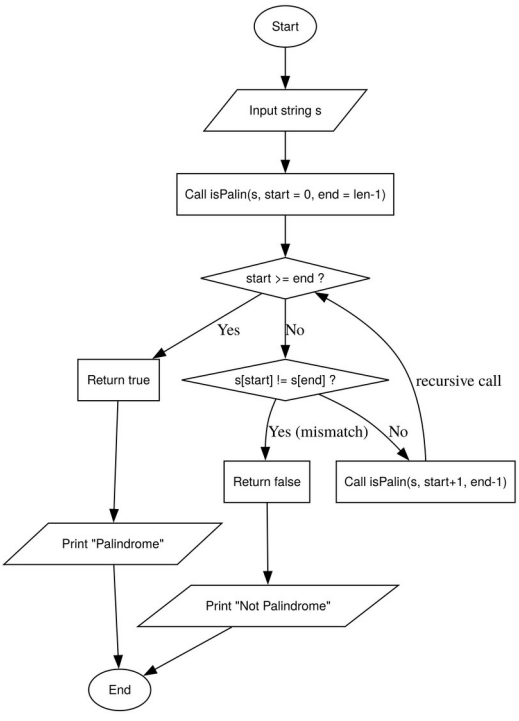

```
void print_output_rev(string s, int n) {
    if (n == 0) return;
    cout << s[n-1] << endl;
    print_output_rev(s, n-1);
}
```

Question 3



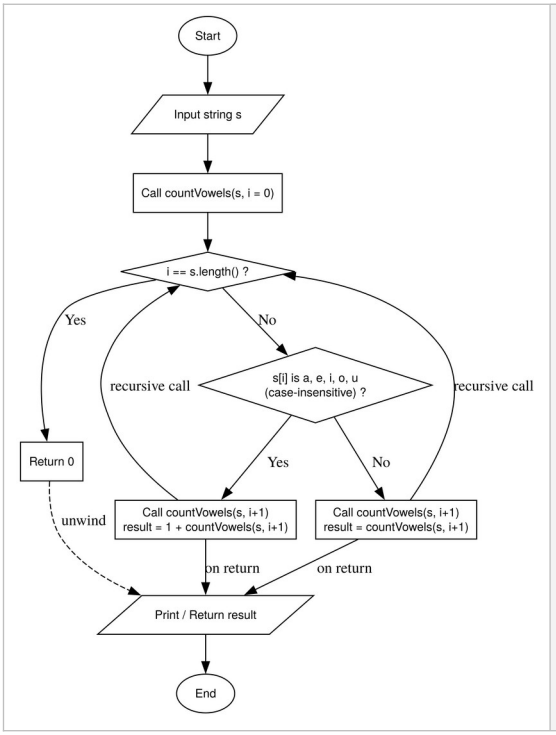

```
void rev_string(string s, int len) {
    if (len == 0) return;
    cout << s[len - 1];
    rev_string(s, len-1);
}
```

Question 4




```
bool palindrome(string s, int i = 0) {
    // false when s[left] != s[right,
    if ( i >= s.length() / 2) return true;
    if (s[i] != s[s.length()- i -1]) return false;
    else return palindrome(s, i+1);
}
```

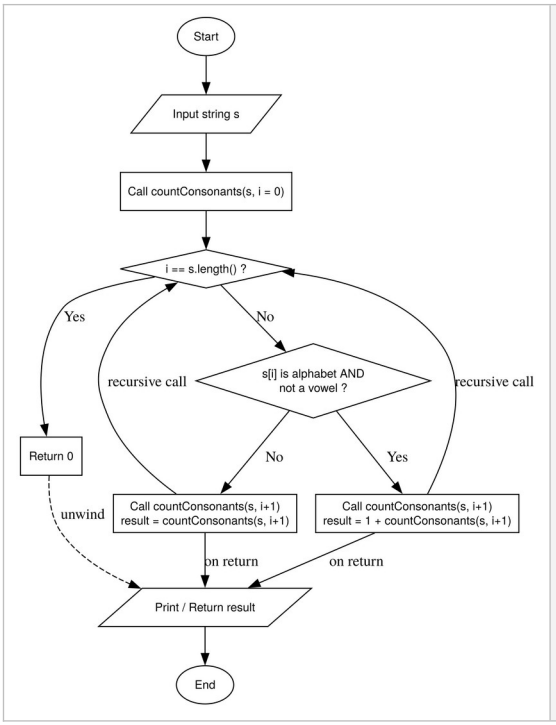
Question 5




```
int count_vowels(string s, int n) {
        if (n == 0) return  0;

        char c = s[n-1];
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c ==
'u' || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U')
         {
            return 1 + count_vowels(s,  n-1);
         } else {
            return 0 + count_vowels(s, n-1);
         }
}
```

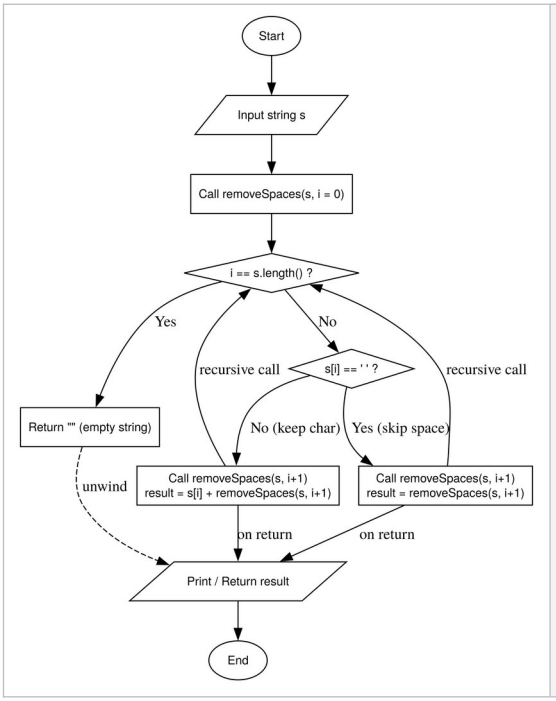
Question 6




```
bool vowel(char c) {
    return ((c == 'a' || c == 'e' || c == 'i' || c == 'o' || c
== 'u' || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c ==
'U'));
}

int consonants(string s, int n) {
    if (n == 0) return 0;
    if (!vowel(s[n-1])) return 1 + consonants(s,  n-1);
    else return 0 + consonants(s, n-1);
}
```

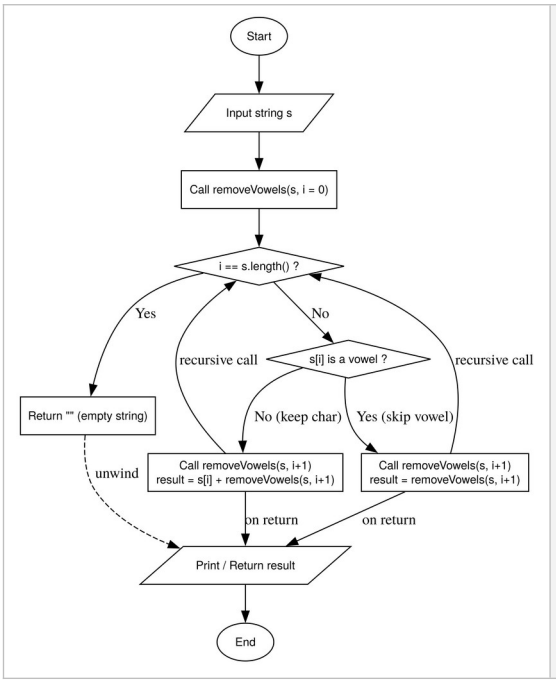
Question 7




```
string remove_spaces(string s, int n) {
    if (n == 0) return "";

    if (s[n-1] == ' ') return  remove_spaces(s, n-1);
    else return  remove_spaces(s, n-1) + s[n-1];
}
```

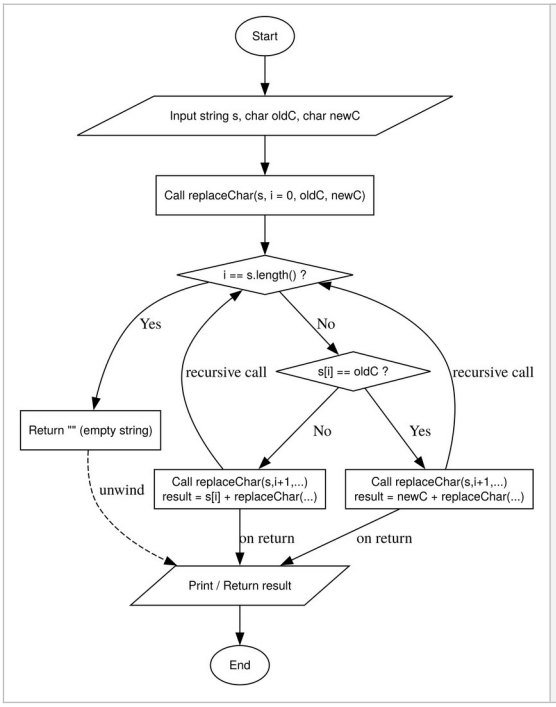
Question 8




```
bool vowel(char c) {
    return ((c == 'a' || c == 'e' || c == 'i' || c == 'o' || c
== 'u' || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c ==
'U'));
}

string remove_vowel(string s, int n) {
    if (n == 0) return "";
    if (vowel(s[n-1])) return remove_vowel(s, n-1);
    return remove_vowel(s,  n-1) + s[n-1];
}
```

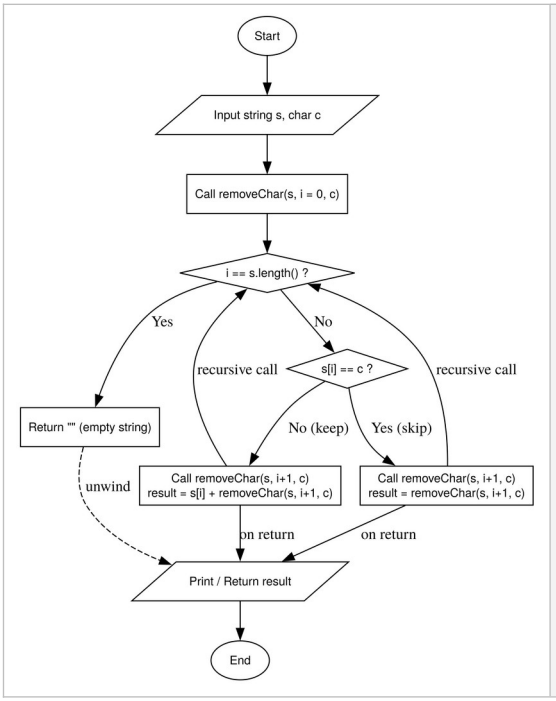
Question 9




```
string replace_occur(string s, char c, char r, int n) {
    if (n == 0) return "";

    if (s[n-1] == c) return replace_occur(s, c, r, n-1) + r;
    return replace_occur(s,  c,  r, n-1) + s[n-1];
}
```

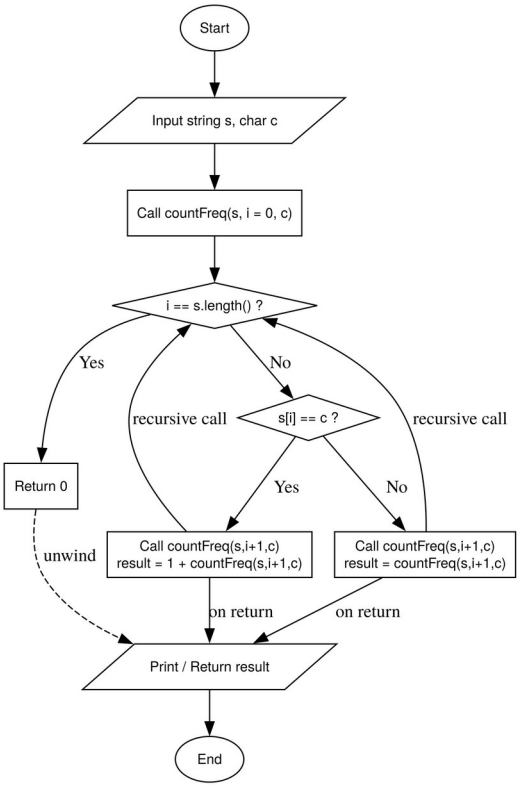
Question 10




```
string remove_occur(string s, char c, int n) {
    if (n==0) return "";

    if (s[n-1] == c) return remove_occur(s, c, n-1);
    return remove_occur(s, c, n-1) + s[n-1];
}
```

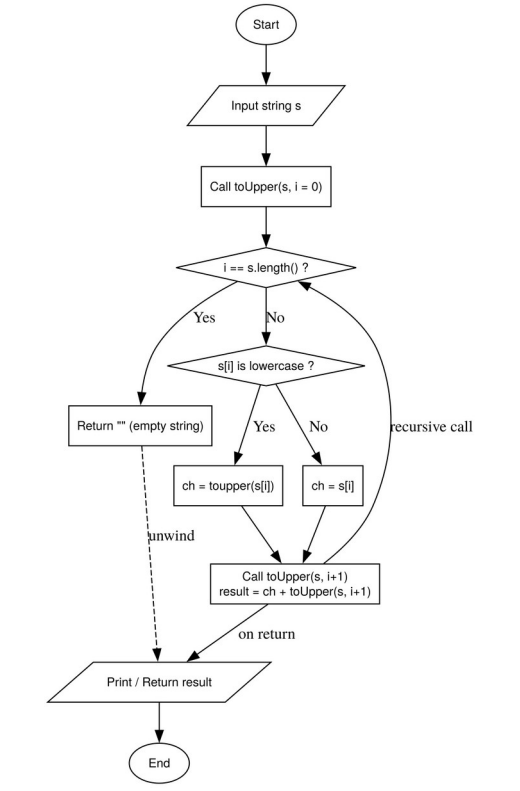
Question 11




```
int count_freq(string s, char c, int n) {
    if (n == 0) return 0;

    if (s[n-1] == c) return 1 + count_freq(s, c, n-1);
    else return 0 + count_freq(s, c, n-1);
}
```

Question 12

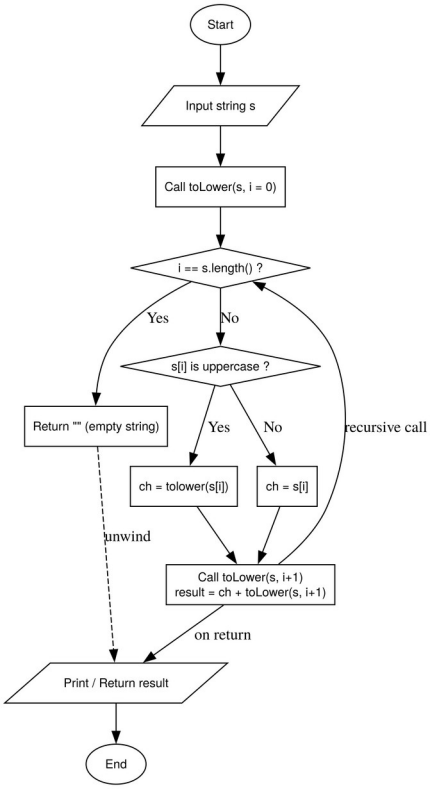



```
string to_uppercase(string s, int n) {
    if (n == 0) return "";

    char current = s[n-1];
    if (current >= 'a' && current <= 'z') current = current -
32;

    return to_uppercase(s, n-1) + current;
}
```

Question 13

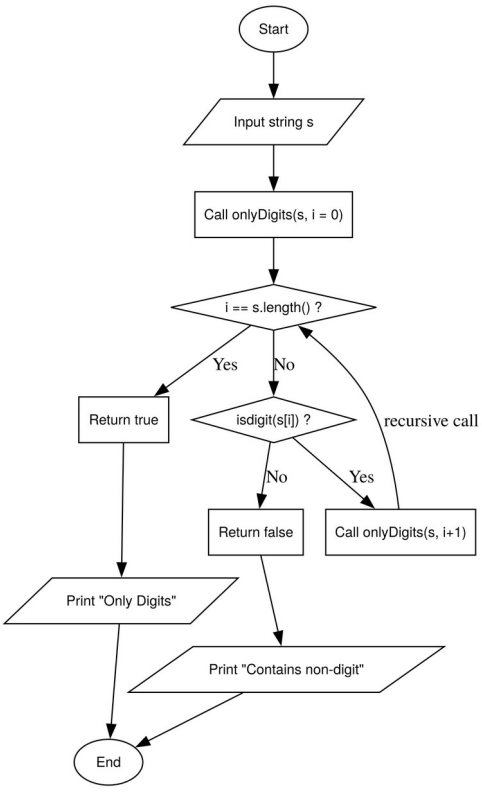



```
string to_lowercase(string s, int n) {
    if (n == 0) return "";

    char current = s[n-1];
    if (current >= 'A' && current <= 'Z') current = current +
32;

    return to_lowercase(s, n-1) + current;
}
```

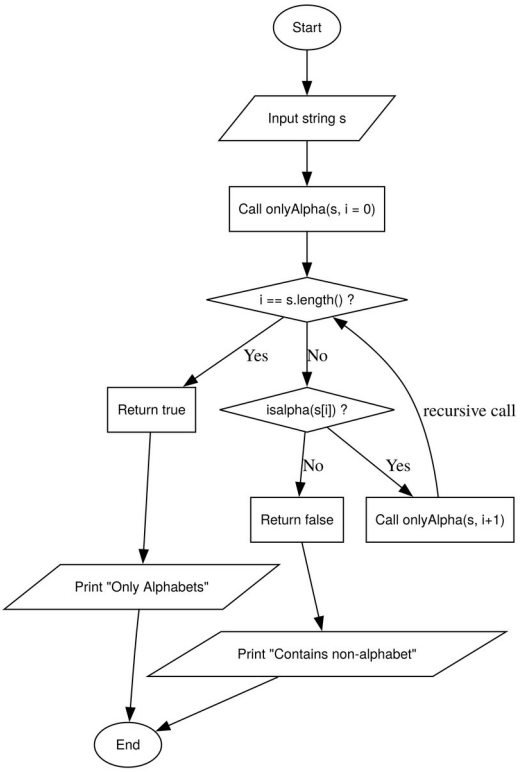
Question 14




```
bool only_digits(string s, int n) {
    if (n == 0) return true;

    if (s[n-1] >= '0' && s[n-1] <= '9') return only_digits(s,
n-1);
    return false;
}
```

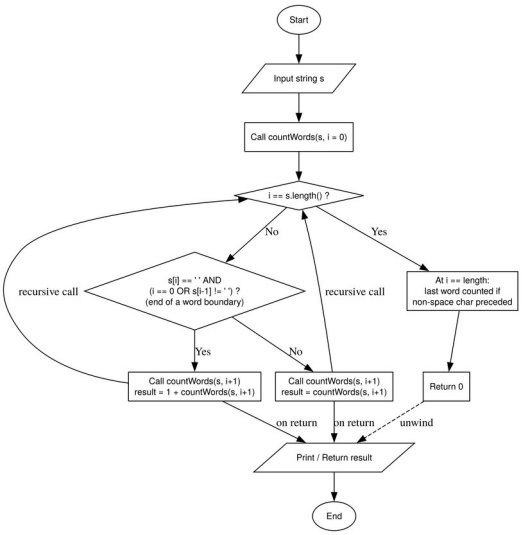
Question 15




```
bool only_alphabets(string s, int n) {
    if (n == 0) return true;

    char c = s[n-1];
    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) return
only_alphabets(s, n-1);
    return false;
}
```

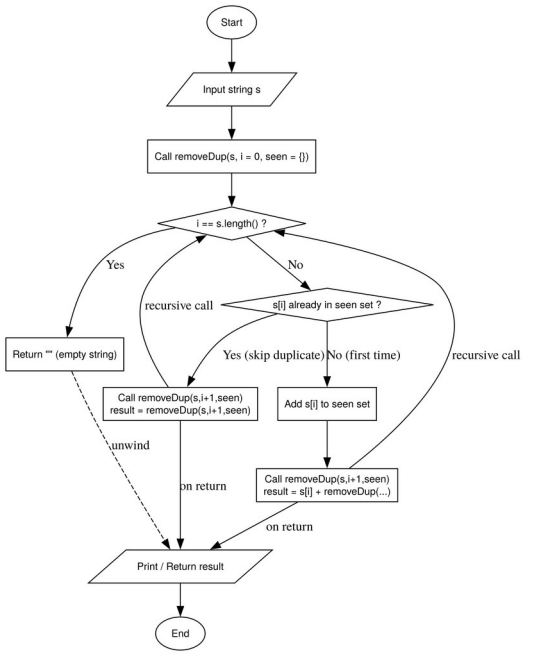
Question 16




```
int count_words(string s, int n) {
    if (n == 0) return 0;

    if (s[n-1] != ' ' && (n == 1 || s[n-2] == ' ')) return 1 +
count_words(s, n-1);
    return 0 + count_words(s, n-1);
}
```

Question 17

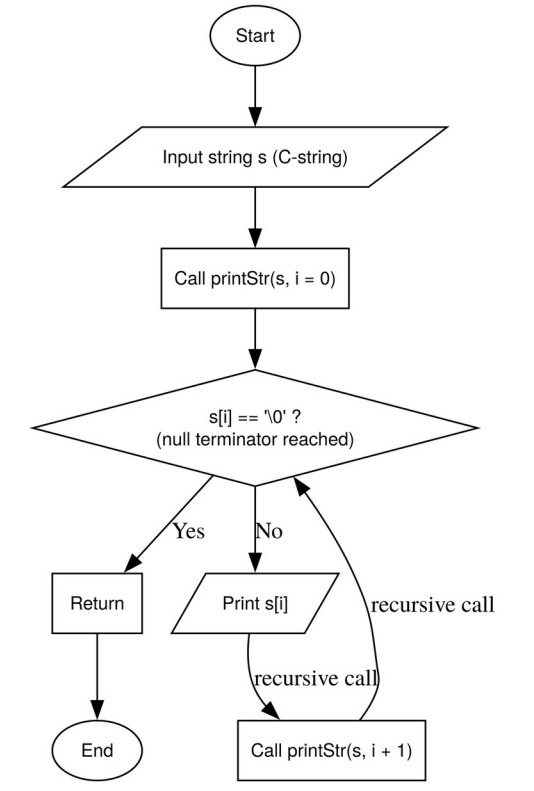



```
bool char_exists(string s, char c, int n) {
    if (n == 0) return false;
    if (s[n-1] == c) return true;
    return char_exists(s, c, n-1);
}

string remove_duplicates(string s, int n) {
    if (n == 0) return "";

    if (char_exists(s, s[n-1], n-1)) return remove_duplicates(s,
n-1);
    return remove_duplicates(s, n-1) + s[n-1];
}
```

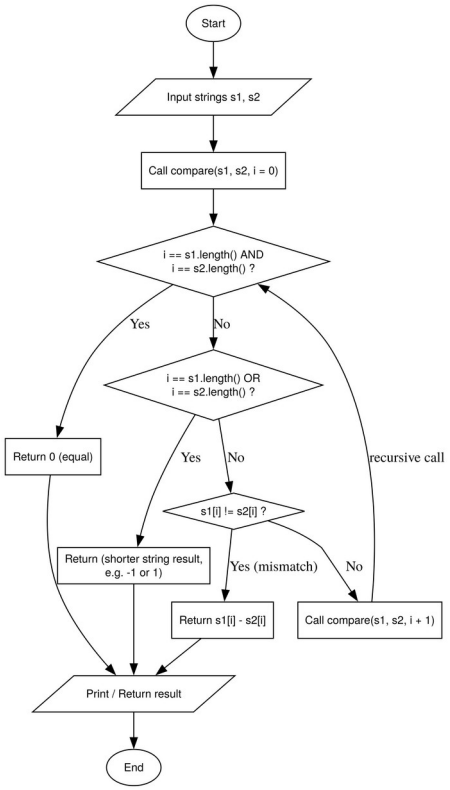
Question 18




```
void print2_string(string s, int i = 0) {
    if (s[i] == '\0') return;

    cout << s[i];
    print2_string(s, i + 1);
}
```

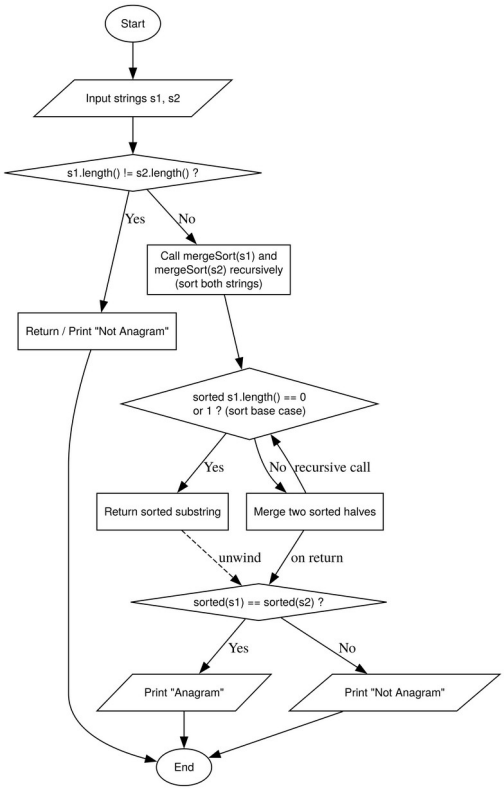
Question 19




```
bool compare_strings(string s1, string s2, int n1, int n2) {
    if (n1 == 0 && n2 == 0) return true;
    if (n1 == 0 || n2 == 0) return false;

    if (s1[n1-1] != s2[n2-1]) return false;
    return compare_strings(s1, s2, n1-1, n2-1);
}
```

Question 20




```
bool presence_check(string &s, char c, int i = 0) {
    if (i == s.length()) return false;

    if (s[i] == c) {
        s[i] = '0'; // this shte flag that we use to mark that
this place hath been checked
        return true;
    }
    return presence_check(s,  c, i+1);

}

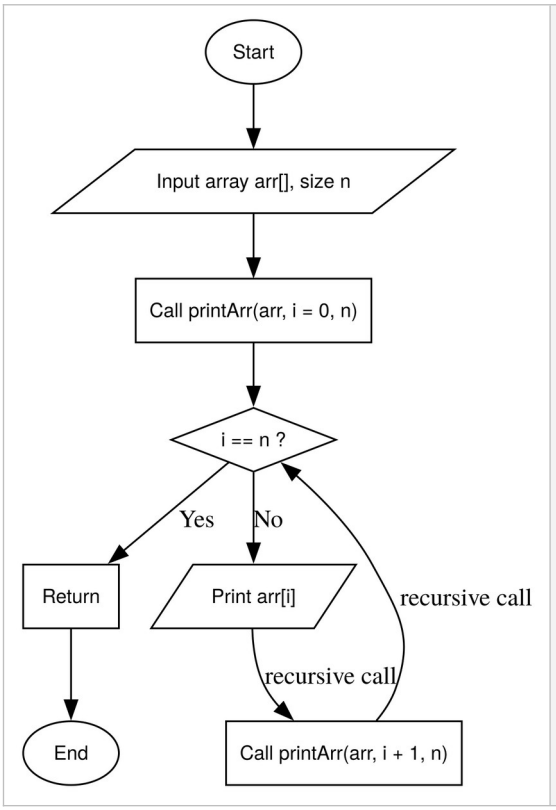
bool check_anagram(string x, string y, int i = 0) {
    if (x.length() != y.length()) return false;

    if (i == x.length()) return true;

    if (!presence_check(y, x[i])) {
        return false;
    }
    return check_anagram(x, y, i+1);
}
```

Phase 6 – Array Recursion (No Loops Allowed)

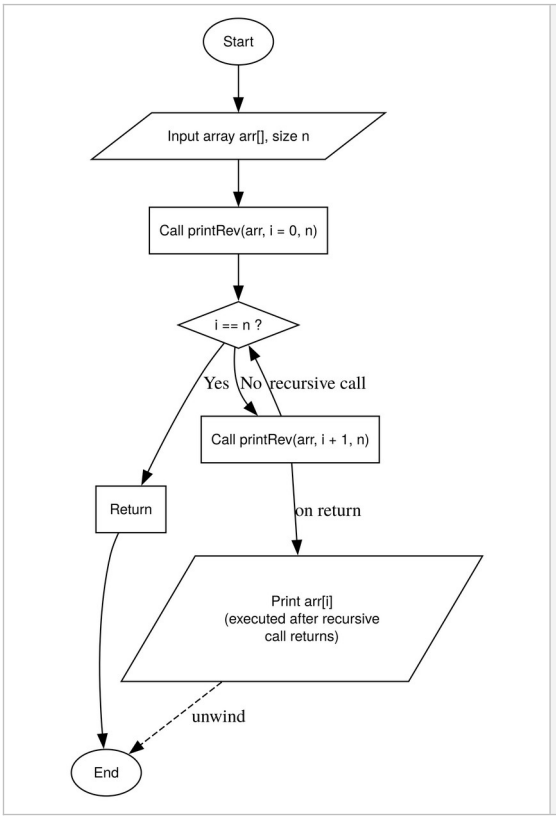
Question 1




```
void print_elements(int a[], int n) {
    if (n == 0) return;

    print_elements(a, n-1);
    cout << a[n-1] << " ";
}
```

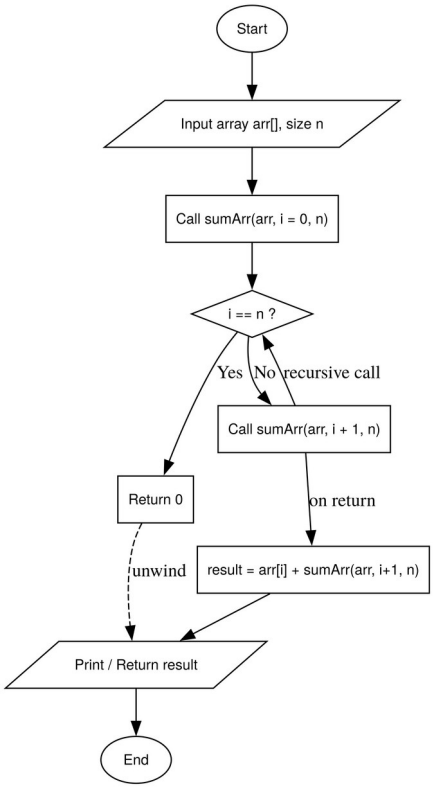
Question 2




```
void print_elements_rev(int a[], int n) {
    if (n == 0) return;

    cout << a[n-1] << " ";
    print_elements_rev(a, n-1);
}
```

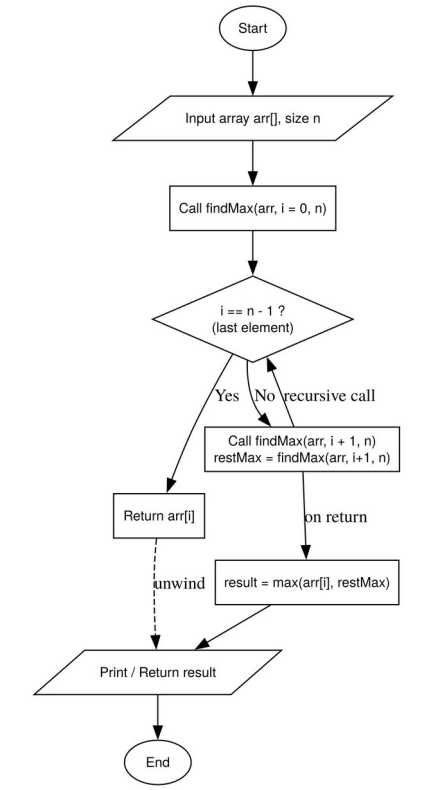
Question 3




```
ll arr_sum(int a[], int n) {
    if (n == 0) return 0;

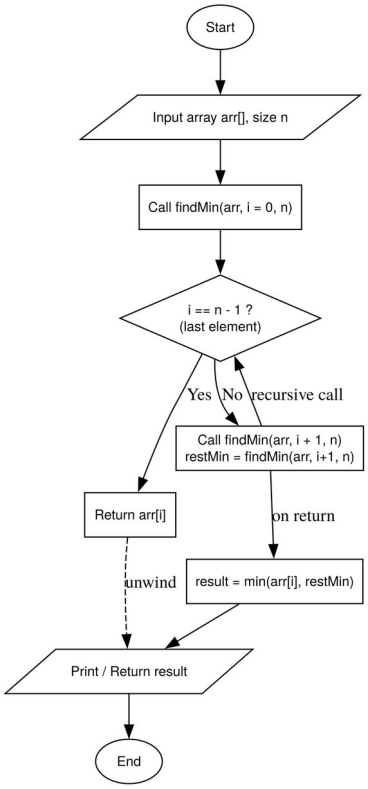
    return a[n-1] + arr_sum(a, n-1);
}
```

Question 4




```
int find_max(int arr[], int n) {
    if (n == 1) return arr[0];
    int max_rest = find_max(arr, n - 1);
    if (arr[n - 1] > max_rest) return arr[n - 1];
    return max_rest;
}
```

Question 5

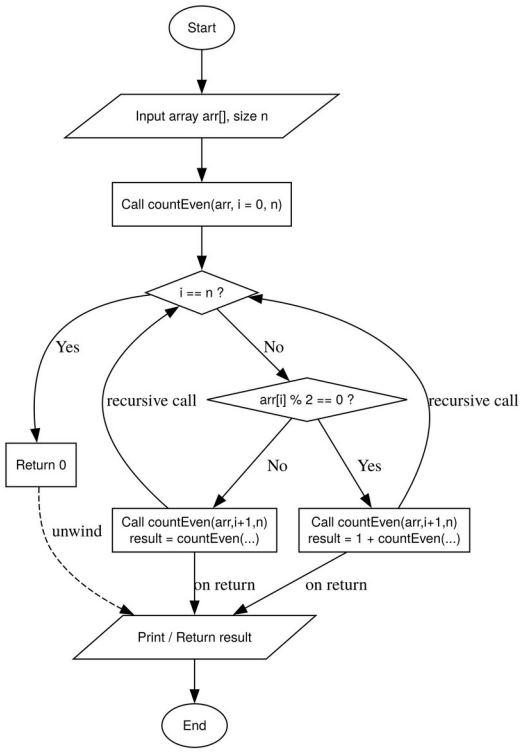



```
int find_min(int arr[], int n) {
    if (n == 1) return arr[0];

    int min_rest = find_min(arr, n - 1);

    if (arr[n - 1] < min_rest) return arr[n - 1];
    return min_rest;
}
```

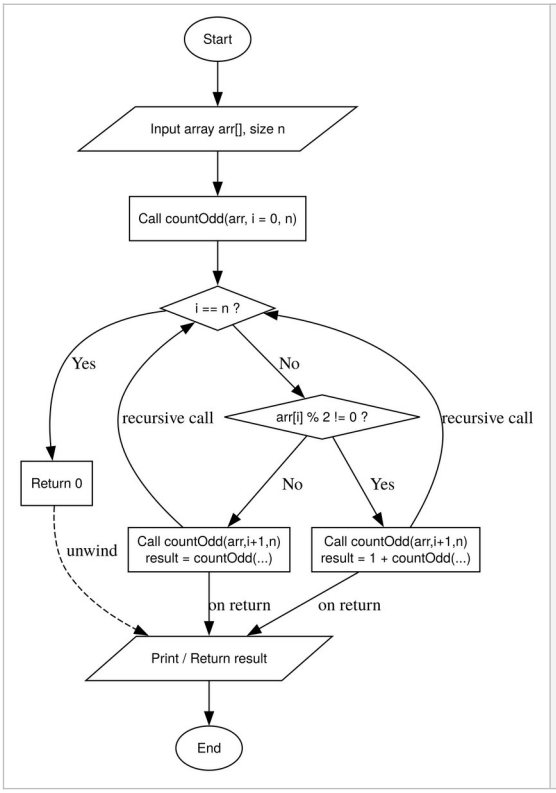
Question 6




```
int count_even(int a[], int n) {
    if (n == 0) return 0;

    if (a[n-1] % 2 == 0) return 1 + count_even(a, n-1);
    return 0 + count_even(a, n-1);
}
```

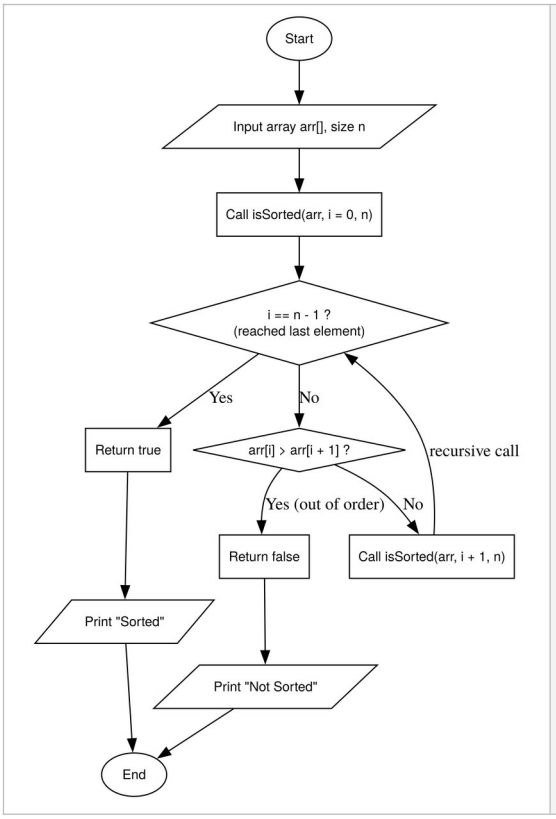
Question 7




```
int count_odd(int a[], int n) {
    if (n == 0) return 0;

    if (a[n-1] % 2 != 0) return 1 + count_odd(a, n-1);
    return 0 + count_odd(a, n-1);
}
```

Question 8

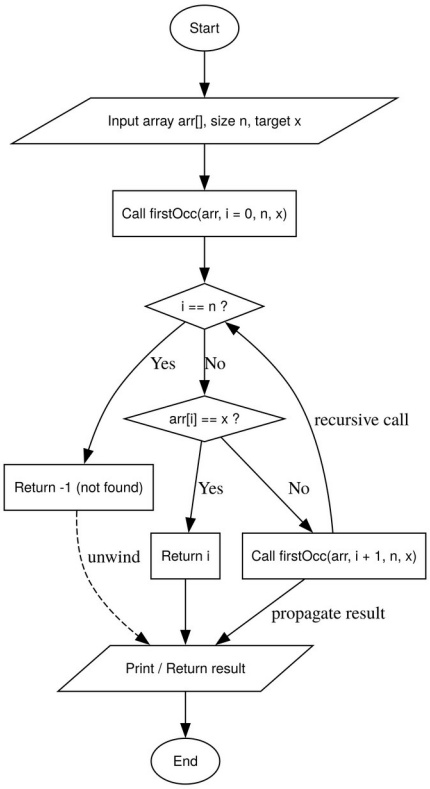



```
bool check_for_asc(int a[], int n) {
    if (n == 0 || n == 1) return true;

    if (a[n-1] < a[n-2]) return false;

    return check_for_asc(a, n-1);
}
```

Question 9

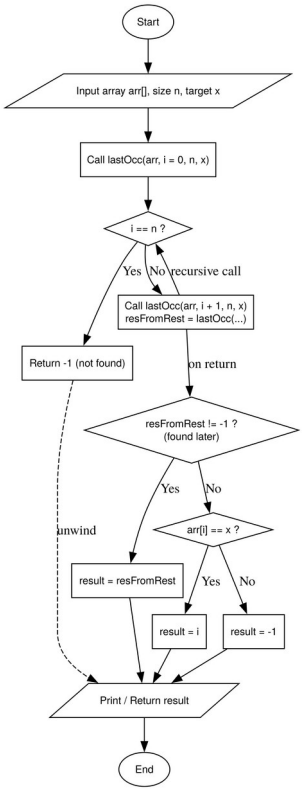



```
int first_occurence(int a[], int n, int x, int i = 0) {
    if (i == n) return -1;

    if (a[i] == x) return i;

    return first_occurence(a, n, x, i+1);
}
```

Question 10

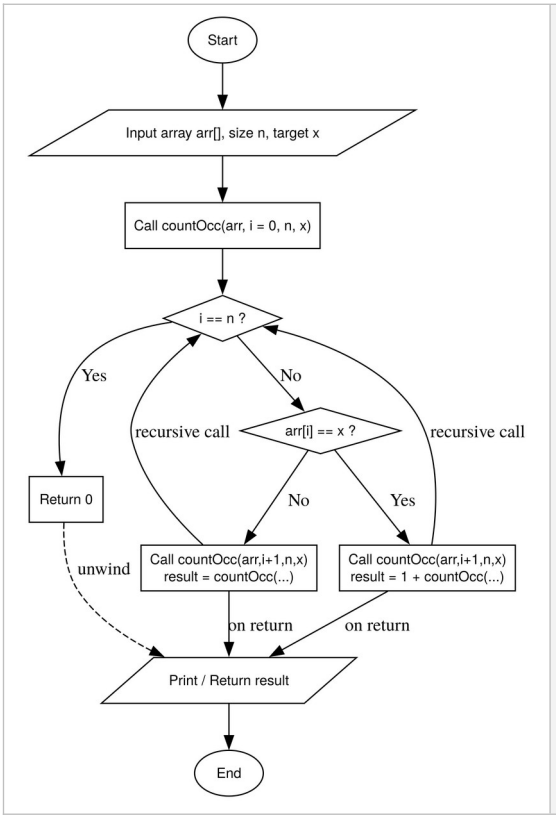



```
int last_occurence(int a[], int n, int x) {
    if (n ==0) return -1;

    if (a[n-1] == x) return n-1;
    return last_occurence(a, n-1, x);

}
```

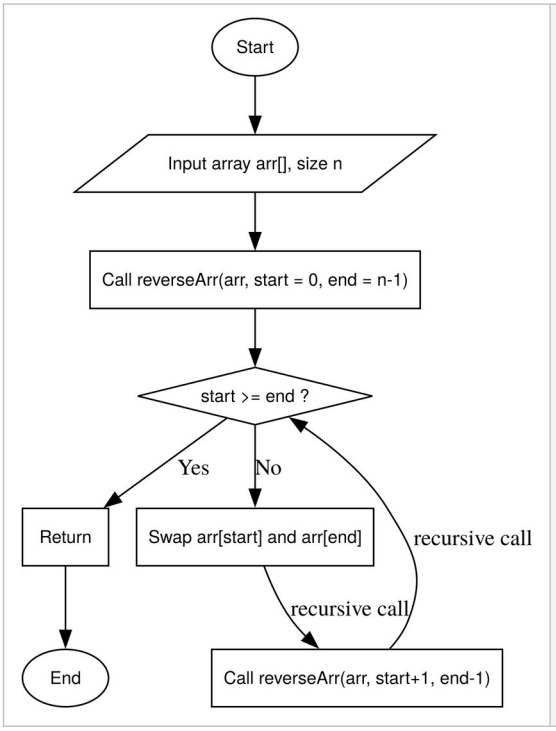
Question 11




```
int tot_occurances(int a[], int n, int c) {
    if (n == 0) return 0;

    if (a[n-1] == c) return 1 + tot_occurances(a, n-1, c);
    return 0 + tot_occurances(a, n-1, c);
}
```

Question 12

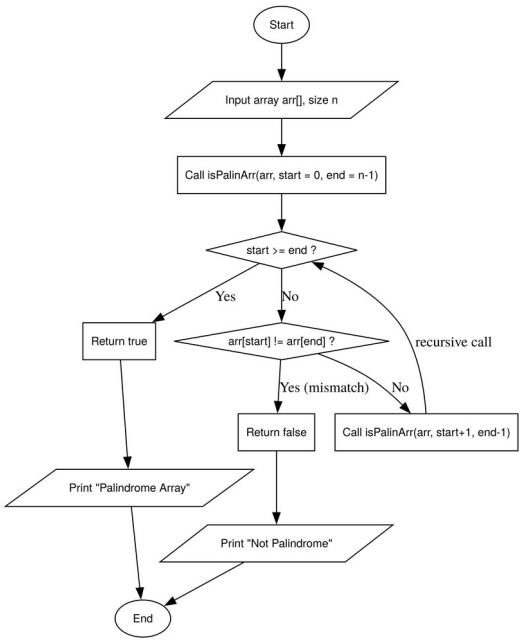



```
void rev_array(int a[], int n, int i = 0) {
    if (i >= n / 2) return ;

    int temp = a[i];
    a[i] = a[n - i - 1];
    a[n - i - 1] =  temp;

    rev_array(a, n, i + 1);
}
```

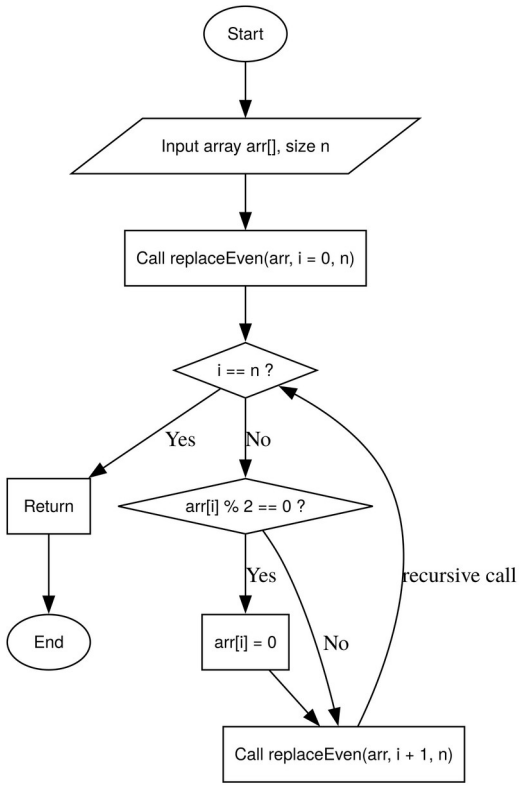
Question 13




```
bool palindrome(int a[], int n, int i= 0 ) {
    if (i >= n / 2) return true;

    if (a[i] != a[n - i -1]) return false;
    return palindrome(a, n, i+1);
}
```

Question 14

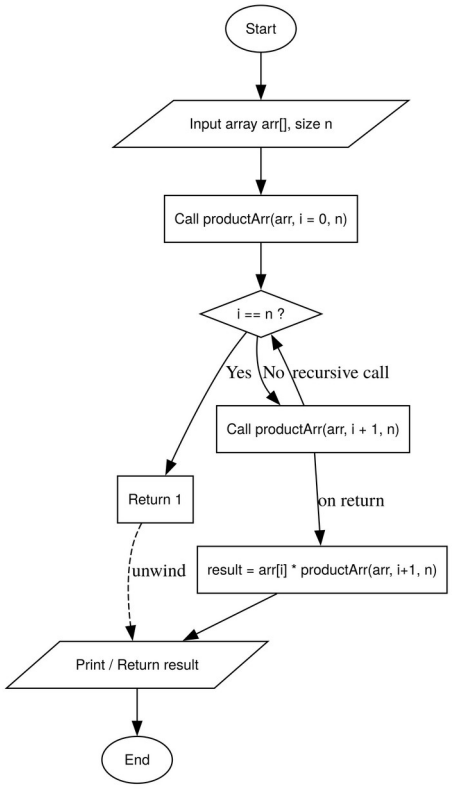



```
void replace_even_zero(int a[], int n) {
    if (n == 0) return;

    if (a[n-1] % 2 == 0) {
        a[n-1] = 0;
    }

    replace_even_zero(a, n-1);
}
```

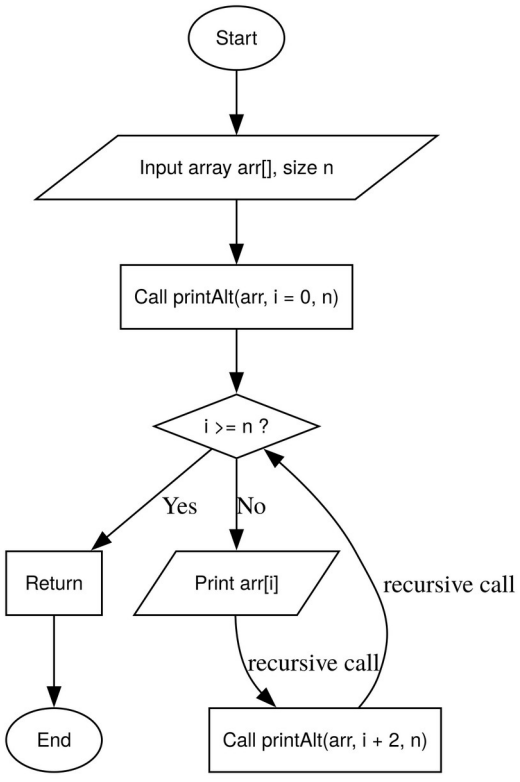
Question 15




```
ll  arr_product(int a[], int n) {
    if (n == 0) return 1;

    return a[n-1] * arr_product(a, n-1);
}
```

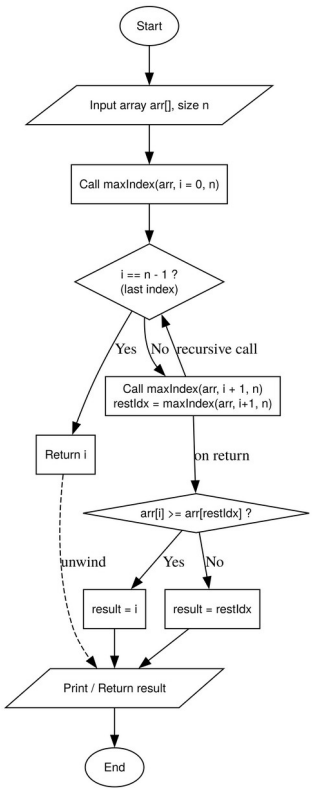
Question 16




```
void print_alternate(int a[], int n, int i = 0) {
    if (i >= n) return;
    cout << a[i] << " ";

    print_alternate(a, n, i + 2);
}
```

Question 17

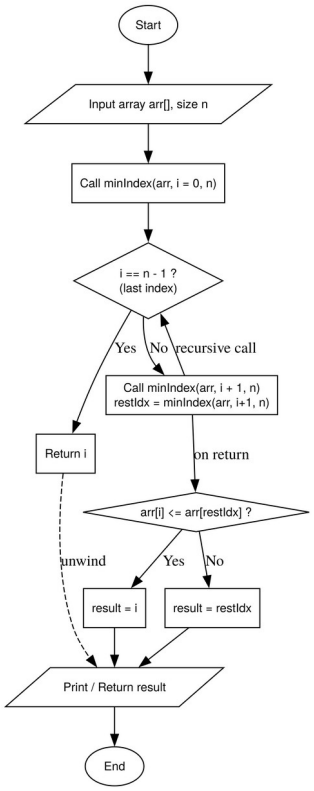



```
int find_max_index(int a[], int n) {
    if (n == 1) return 0;

    int max_idx_rest = find_max_index(a, n - 1);

    if (a[n - 1] > a[max_idx_rest]) return n - 1;
    return max_idx_rest;
}
```

Question 18

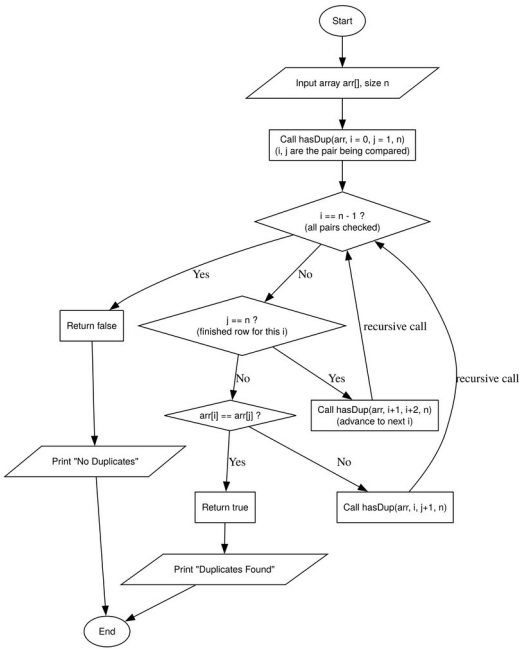



```
int find_min_index(int a[], int n) {
    if (n == 1) return 0;

    int min_idx_rest = find_min_index(a, n - 1);

    if (a[n - 1] < a[min_idx_rest]) return n - 1;
    return min_idx_rest;
}
```

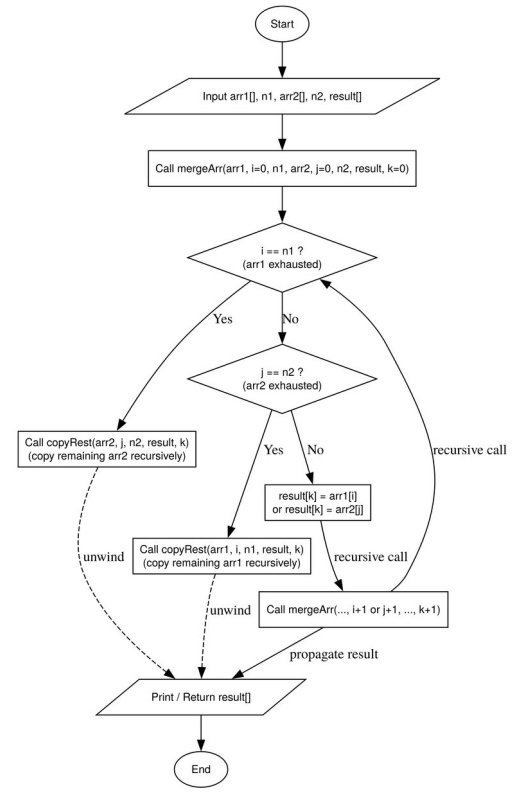
Question 19




```
bool exists_in_array(int a[], int n, int target) {
    if (n == 0) return false;
    if (a[n - 1] == target) return true;
    return exists_in_array(a, n - 1, target);
}

bool has_duplicates(int a[], int n) {
    if (n <= 1) return false;
    if (exists_in_array(a, n - 1, a[n - 1])) return true;
    return has_duplicates(a, n - 1);
}
```

Question 20




```
void merge_arrays(int a[], int n1, int b[], int n2, int
merged[], int i = 0) {
    if (i == n1 + n2) return;

    if (i < n1) {
        merged[i] = a[i];
    }
    else {
        merged[i] = b[i - n1];
    }
    merge_arrays(a, n1, b, n2, merged, i+1);
}
```